

Analisis Cluster

Ferdian Bangkit Wijaya, S.Stat., M.Si.

Section 1

Pengantar Clustering

Konsep Jarak

Konsep Dasar

- Bertujuan untuk **mengelompokkan objek-objek** (bisa berupa individu, produk, wilayah, atau entitas lainnya) ke dalam beberapa kelompok (klaster)
- Pengelompokan ini didasarkan pada **kesamaan karakteristik** yang dimiliki oleh objek-objek tersebut, berdasarkan beberapa variabel yang diukur secara **bersamaan**
- Objek-objek dalam **satu klaster akan semirip mungkin** satu sama lain, sementara objek-objek di **klaster yang berbeda akan sebeda mungkin**
- Proses ini dilakukan tanpa menggunakan kategori atau label yang sudah ada sebelumnya (**unsupervised**), sehingga sangat berguna untuk menemukan struktur atau pola tersembunyi dalam data

Karakteristik Clustering

- **Variabel Multivariat:**

Menggunakan beberapa variabel secara **bersamaan** untuk mengukur karakteristik objek. Variabel-variabel ini bisa bersifat **kontinu** (misalnya, tinggi badan, pendapatan), **diskrit** (misalnya, jumlah anak), atau **kategorikal** (misalnya, jenis kelamin, tingkat pendidikan).

- **Ukuran Jarak atau Kemiripan:**

Untuk mengelompokkan objek, kita perlu mengukur seberapa **dekat atau mirip** objek-objek tersebut

Konsep Jarak

- Jarak (distance) atau ketidakmiripan (dissimilarity) adalah ukuran kuantitatif yang menggambarkan seberapa **berbeda** dua objek atau titik data satu sama lain.
- Kebalikan dari kemiripan (similarity), yang mengukur seberapa "dekat" atau "serupa" dua objek.
- Tujuannya adalah mengelompokkan objek-objek yang memiliki jarak dekat (mirip) dan memisahkan objek-objek yang memiliki jarak jauh (berbeda).
- Pemilihan perhitungan jarak menyesuaikan **tipe data** yang dimiliki.

Jarak untuk Data Numerik (Kontinu/ Diskret) (Interval & Rasio)

- Jarak Euclidian
- Jarak Manhattan
- Jarak Minkowski
- Jarak Chebyshev
- Jarak Mahalanobis

Jarak untuk Data Kategorik (Nominal & Ordinal)

- Jaccard Coefficient (Nominal)
- Jarak Hamming (Nominal)
- Matching Coefficient (Nominal)
- Jarak berbasis Rank (Ordinal)

Jarak untuk Data Campuran (Kategorik dan Numerik)

- Jarak Gower

Jarak Euclidian

- Yaitu jarak garis lurus antara dua titik dalam ruang multi-dimensi (banyak objek dan banyak variabel).
- Persis seperti mengukur jarak antara dua lokasi di peta menggunakan penggaris (Konsep Pythagoras).
- Misal, jika memiliki dua objek (atau titik data), sebut saja p dan q. Masing-masing objek ini memiliki n karakteristik (variabel). Jadi, dapat ditulis p sebagai $(p_1, p_2, \dots, p_n)$ dan q sebagai $(q_1, q_2, \dots, q_n)$.
- Rumus Jarak Euclidian :

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2} = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Jarak Euclidian

Siswa	Tinggi (cm) (X_1)	Berat (kg) (X_2)	Nilai Mtk (X_3)
A	170	65	85
B	180	75	70
C	172	68	88

Hal yang sama dilakukan untuk menghitung jarak :
A dengan C
B dengan C

Jarak antara Objek/ Siswa A dengan B adalah

- Objek p = Siswa A = (170, 65, 85)
- Objek q = Siswa B = (180, 75, 70)
- $p_1 = 170, q_1 = 180$
- $p_2 = 65, q_2 = 75$
- $p_3 = 85, q_3 = 70$

$$d(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + (p_3 - q_3)^2}$$

$$d(A, B) = \sqrt{(170 - 180)^2 + (65 - 75)^2 + (85 - 70)^2}$$

$$d(A, B) = \sqrt{(-10)^2 + (-10)^2 + (15)^2}$$

$$d(A, B) = \sqrt{100 + 100 + 225}$$

$$d(A, B) = \sqrt{425}$$

$$d(A, B) \approx 20.62$$

Jarak Manhattan

- Jarak Manhattan, sering juga disebut Jarak City Block, Jarak L1 , atau Jarak Taksi (Taxi-cab), adalah cara lain untuk mengukur jarak antara dua titik dalam ruang multi-dimensi.
- Berbeda dengan Jarak Euclidean yang mengambil "jalan pintas" lurus, Jarak Manhattan mengukur jarak seolah-olah berjalan di sepanjang grid perkotaan, di mana hanya bisa bergerak secara vertikal atau horizontal (tidak bisa diagonal).
- Rumus Jarak Manhattan :

$$d(p, q) = |p_1 - q_1| + |p_2 - q_2| + \dots + |p_n - q_n| = \sum_{i=1}^n |p_i - q_i|$$

Jarak Manhattan

Siswa	Tinggi (cm) (X_1)	Berat (kg) (X_2)	Nilai Mtk (X_3)
A	170	65	85
B	180	75	70
C	172	68	88

Jarak antara Objek/ Siswa A dengan B adalah

- Objek p = Siswa A = (170, 65, 85)
- Objek q = Siswa B = (180, 75, 70)

$$d(A, B) = |170 - 180| + |65 - 75| + |85 - 70|$$

$$d(A, B) = |-10| + |-10| + |15|$$

$$d(A, B) = 10 + 10 + 15$$

$$d(A, B) = 35$$

Hal yang sama dilakukan untuk menghitung jarak :

A dengan C

B dengan C

Jarak Minkowski

- Jarak Minkowski merupakan generalisasi dari jarak Euclidian dan Manhattan.
- Rumus Jarak Minkowski :

$$d(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^h \right)^{1/h}$$

Parameter kunci adalah “h”, terkadang disebut p atau λ , yang merupakan bilangan real ≥ 1

- Jika $h=1$, maka akan menjadi jarak Manhattan.
- Jika $h=2$, maka akan menjadi jarak Euclidian.
- Jika $h=\infty$, maka akan menjadi jarak Chebyshev

Jarak Minkowski

Siswa	Tinggi (cm) (X_1)	Berat (kg) (X_2)	Nilai Mtk (X_3)
A	170	65	85
B	180	75	70
C	172	68	88

Jarak antara Objek/ Siswa A dengan B, dengan $h=3$ adalah

- Objek p = Siswa A = (170, 65, 85)
- Objek q = Siswa B = (180, 75, 70)

$$d(A, B) = (|170 - 180|^3 + |65 - 75|^3 + |85 - 70|^3)^{1/3}$$

$$d(A, B) = (|-10|^3 + |-10|^3 + |15|^3)^{1/3}$$

$$d(A, B) = (10^3 + 10^3 + 15^3)^{1/3}$$

$$d(A, B) = (1000 + 1000 + 3375)^{1/3}$$

$$d(A, B) = (5375)^{1/3}$$

$$d(A, B) \approx 17.52$$

Hal yang sama dilakukan untuk menghitung jarak :

A dengan C

B dengan C

Jarak Chebyshev

- Jarak Chebyshev, sering juga disebut Jarak Papan Catur (Chessboard Distance) atau Jarak L^∞ , adalah metrik jarak yang mengukur "langkah" terbesar yang diperlukan untuk menyamakan dua titik di sepanjang salah satu dimensi.
- Bayangkan sebuah raja di papan catur; ia dapat bergerak satu langkah ke segala arah (horizontal, vertikal, atau diagonal).
- Rumus Jarak Chebyshev:

$$d(p, q) = \max_i (|p_i - q_i|)$$

Jarak Chebyshev

Siswa	Tinggi (cm) (X_1)	Berat (kg) (X_2)	Nilai Mtk (X_3)
A	170	65	85
B	180	75	70
C	172	68	88

Jarak antara Objek/ Siswa A dengan B adalah

Hitung perbedaan absolut untuk setiap dimensi:

- $|170 - 180| = |-10| = 10$
- $|65 - 75| = |-10| = 10$
- $|85 - 70| = |15| = 15$

$$d(A, B) = \max(10, 10, 15)$$

$$d(A, B) = 15$$

Hal yang sama dilakukan untuk menghitung jarak :

A dengan C

B dengan C

Jarak Mahalanobis

- Jarak Mahalanobis mengukur jarak suatu titik dari "pusat" distribusi data (biasanya mean) dalam satuan standar deviasi yang telah disesuaikan dengan korelasi.
- Jarak Mahalanobis (biasanya dinyatakan sebagai kuadratnya, D^2) antara suatu titik data $x=(x_1, x_2, \dots, x_n)$ dan pusat distribusi (mean) $\mu=(\mu_1, \mu_2, \dots, \mu_n)$ dihitung menggunakan invers dari matriks kovariansi (S^{-1}).
- Rumus Jarak Mahalanobis :

$$D^2(x, \mu) = (x - \mu)^T S^{-1} (x - \mu)$$

Jarak Mahalanobis

Siswa	Tinggi (cm) (X_1)	Berat (kg) (X_2)	Nilai Mtk (X_3)
A	170	65	85
B	180	75	70
C	172	68	88
D	175	70	80

Notes : jumlah data > variabel

Jarak antara Objek/ Siswa A ke pusat Data adalah

Langkah 1: Hitung Vektor Mean (μ)

- $\mu_1 = (170 + 180 + 172 + 175)/4 = 697/4 = 174.25$
- $\mu_2 = (65 + 75 + 68 + 70)/4 = 278/4 = 69.5$
- $\mu_3 = (85 + 70 + 88 + 80)/4 = 323/4 = 80.75$
- $\mu = \begin{pmatrix} 174.25 \\ 69.5 \\ 80.75 \end{pmatrix}$

Langkah 2: Hitung Matriks Kovariansi (S)

Selisih tiap data dengan mean ($N - 1 = 3$):

- A: (-4.25, -4.5, 4.25)
- B: (5.75, 5.5, -10.75)
- C: (-2.25, -1.5, 7.25)
- D: (0.75, 0.5, -0.75)

Menghitung varians dan kovarians:

- $S_{11} = \frac{(-4.25)^2 + 5.75^2 + (-2.25)^2 + 0.75^2}{3} \approx 18.92$
- $S_{22} = \frac{(-4.5)^2 + 5.5^2 + (-1.5)^2 + 0.5^2}{3} \approx 17.67$
- $S_{33} = \frac{4.25^2 + (-10.75)^2 + 7.25^2 + (-0.75)^2}{3} = 62.25$
- $S_{12} = \frac{(-4.25)(-4.5) + (5.75)(5.5) + (-2.25)(-1.5) + (0.75)(0.5)}{3} \approx 18.17$
- $S_{13} = \frac{(-4.25)(4.25) + (5.75)(-10.75) + (-2.25)(7.25) + (0.75)(-0.75)}{3} \approx -32.25$
- $S_{23} = \frac{(-4.5)(4.25) + (5.5)(-10.75) + (-1.5)(7.25) + (0.5)(-0.75)}{3} \approx -29.83$

Jarak Mahalanobis

Matriks Kovariansi S :

$$S \approx \begin{pmatrix} 18.92 & 18.17 & -32.25 \\ 18.17 & 17.67 & -29.83 \\ -32.25 & -29.83 & 62.25 \end{pmatrix}$$

Langkah 3: Hitung Invers Matriks Kovariansi (S^{-1})

$$S^{-1} = \frac{1}{\det(S)} \text{adj}(S)$$

$$S^{-1} \approx \begin{pmatrix} 0.618 & -0.596 & 0.170 \\ -0.596 & 0.635 & -0.198 \\ 0.170 & -0.198 & 0.063 \end{pmatrix}$$

Langkah 4: Hitung Jarak Mahalanobis $D^2(A, \mu)$

- $x - \mu = \begin{pmatrix} -4.25 \\ -4.5 \\ 4.25 \end{pmatrix}$
- $(x - \mu)^T = (-4.25 \quad -4.5 \quad 4.25)$
- $D^2(A, \mu) = (-4.25 \quad -4.5 \quad 4.25) \begin{pmatrix} 0.618 & -0.596 & 0.170 \\ -0.596 & 0.635 & -0.198 \\ 0.170 & -0.198 & 0.063 \end{pmatrix} \begin{pmatrix} -4.25 \\ -4.5 \\ 4.25 \end{pmatrix}$

$$D^2 \approx 3.79$$

$$D = \sqrt{3.79} \approx 1.95$$

Jarak Mahalanobis dari Siswa A ke pusat kelompok (dengan mempertimbangkan varians dan korelasi) adalah sekitar **1.95**.

Hal yang sama dilakukan untuk Siswa B, C, dan D

Transformasi Data

- Pengukuran Jarak pada Data Numerik, sangat tergantung pada skala yang digunakan.
- Semakin besar skala nya, akan berpengaruh terhadap jarak yang semakin mendominasi.

- Transformasi Data diperlukan

- Transformasi Standarisasi :

$$Z = \frac{X - \mu}{\sigma}$$

- Transformasi Normalisasi (0-1) :

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Jarak Jaccard (Biner : 0/1)

- Indeks Jaccard adalah ukuran kemiripan, dan Jarak Jaccard adalah kebalikannya ($1 - \text{Indeks Jaccard}$).
- Jarak Jaccard mengabaikan kecocokan 0-0 (M_{00}). Ini membuatnya sangat cocok untuk data biner asimetris, di mana kehadiran suatu fitur (1) jauh lebih signifikan daripada ketiadaannya (0).
- **Indeks Jaccard (J)** mengukur kemiripan sebagai rasio antara jumlah fitur di mana *kedua* objek bernilai 1 (M_{11}) dengan jumlah fitur di mana *setidaknya salah satu* objek bernilai 1 ($M_{11} + M_{01} + M_{10}$).

Jarak Jaccard (Biner : 0/1)

- Rumus Indeks Jaccard :

$$J(A, B) = \frac{M_{11}}{M_{11} + M_{01} + M_{10}}$$

- Rumus Jarak Jaccard :

$$d_J(A, B) = 1 - J(A, B) = \frac{M_{01} + M_{10}}{M_{11} + M_{01} + M_{10}}$$

Jarak Jaccard (Biner : 0/1)

Pelanggan	Apel (X_1)	Jeruk (X_2)	Pisang (X_3)	Vektor
A	1	0	1	(1, 0, 1)
B	1	1	0	(1, 1, 0)
C	0	0	1	(0, 0, 1)

Jarak A ke B: $d_J(A, B)$

- $A = (1, 0, 1)$
- $B = (1, 1, 0)$
- $M_{11} = 1$
- $M_{01} = 1$
- $M_{10} = 1$
- $M_{00} = 0$ (Tidak ada 0 vs 0)

$$J(A, B) = \frac{1}{1+1+1} = \frac{1}{3}$$
$$d_J(A, B) = 1 - \frac{1}{3} = \frac{2}{3} \approx \mathbf{0.667}$$

Jarak A ke C: $d_J(A, C)$

- $A = (1, 0, 1)$
- $C = (0, 0, 1)$
- $M_{11} = 1$
- $M_{01} = 0$
- $M_{10} = 1$
- $M_{00} = 1$

$$J(A, C) = \frac{1}{1+0+1} = \frac{1}{2} = 0.5$$
$$d_J(A, C) = 1 - 0.5 = \mathbf{0.5}$$

Jarak B ke C: $d_J(B, C)$

- $B = (1, 1, 0)$
- $C = (0, 0, 1)$
- $M_{11} = 0$
- $M_{01} = 1$
- $M_{10} = 2$
- $M_{00} = 0$

$$J(B, C) = \frac{0}{0+1+2} = \frac{0}{3} = 0$$
$$d_J(B, C) = 1 - 0 = \mathbf{1}$$

B dan C tidak memiliki kesamaan sama sekali

Pelanggan A lebih dekat ke Pelanggan C
Pelanggan B lebih dekat ke Pelanggan A
Pelanggan C lebih dekat ke Pelanggan A

Jarak Hamming (Biner & Multinomial)

- Jarak Hamming adalah metrik yang digunakan untuk mengukur perbedaan antara dua **string (atau vektor) dengan panjang yang sama**.
- Metrik ini menghitung **jumlah posisi** di mana kedua string tersebut memiliki simbol yang berbeda.
- Misal memiliki dua objek, A dan B, masing-masing dengan n fitur kategorikal atau biner. Jarak Hamming (d_H) antara A dan B adalah jumlah fitur (i) di mana nilai A_i tidak sama dengan B_i .
- Rumus Jarak Hamming :

$$d_H(A, B) = \sum_{i=1}^n \delta(A_i, B_i)$$

Jarak Hamming (Biner & Multinomial)

Siswa	Ikut Ekskul (X_1)	Jenis Kelamin (X_2)	Lulus Tepat Waktu (X_3)	Vektor
A	Ya (1)	Laki-laki (L)	Ya (1)	(1, L, 1)
B	Ya (1)	Perempuan (P)	Ya (1)	(1, P, 1)
C	Tidak (0)	Perempuan (P)	Ya (1)	(0, P, 1)

Jarak A ke B: $d_H(A, B)$

- $A = (1, L, 1)$
- $B = (1, P, 1)$

1. Posisi 1 (X_1): 1 vs 1 -> Sama (0)
2. Posisi 2 (X_2): L vs P -> Beda (1)
3. Posisi 3 (X_3): 1 vs 1 -> Sama (0)

$$d_H(A, B) = 0 + 1 + 0 = 1$$

Artinya, A dan B berbeda pada 1 posisi.

Jarak A ke C: $d_H(A, C)$

- $A = (1, L, 1)$
- $C = (0, P, 1)$

1. Posisi 1 (X_1): 1 vs 0 -> Beda (1)
2. Posisi 2 (X_2): L vs P -> Beda (1)
3. Posisi 3 (X_3): 1 vs 1 -> Sama (0)

$$d_H(A, C) = 1 + 1 + 0 = 2$$

Artinya, A dan C berbeda pada 2 posisi.

Jarak B ke C: $d_H(B, C)$

- $B = (1, P, 1)$
- $C = (0, P, 1)$

1. Posisi 1 (X_1): 1 vs 0 -> Beda (1)
2. Posisi 2 (X_2): P vs P -> Sama (0)
3. Posisi 3 (X_3): 1 vs 1 -> Sama (0)

$$d_H(B, C) = 1 + 0 + 0 = 1$$

Artinya, B dan C berbeda pada 1 posisi.

Siswa A lebih dekat ke Siswa B daripada ke Siswa C.

Siswa B memiliki kedekatan yang sama ke Siswa A dan Siswa C.

Siswa C lebih dekat ke Siswa B daripada ke Siswa A.

Matching Coefficient (Biner & Multinomial)

- Jarak *Simple Matching Coefficient* (SMC) sebenarnya adalah **ukuran kemiripan**, namun seringkali diubah menjadi **ukuran jarak** (ketidakmiripan) dengan menghitung $1 - \text{SMC}$.
- *Simple Matching Coefficient* (SMC) mengukur proporsi fitur (variabel) yang cocok (baik sama-sama 1 maupun sama-sama 0) antara dua objek.
- Rumus *Simple Matching Coefficient* (SMC) :

$$SMC = \frac{\text{Jumlah variabel yang cocok}}{\text{Total jumlah variabel}} = \frac{M_{11} + M_{00}}{M_{11} + M_{01} + M_{10} + M_{00}}$$

Matching Coefficient (Biner & Multinomial)

Siswa	Ikut Ekskul (X_1)	Jenis Kelamin (X_2)	Lulus Tepat Waktu (X_3)	Vektor
A	Ya (1)	Laki-laki (0)	Ya (1)	(1, 0, 1)
B	Ya (1)	Perempuan (1)	Ya (1)	(1, 1, 1)
C	Tidak (0)	Perempuan (1)	Ya (1)	(0, 1, 1)

Siswa A lebih dekat ke Siswa B daripada ke Siswa C.

Siswa B memiliki kedekatan yang sama ke Siswa A dan Siswa C.

Siswa C lebih dekat ke Siswa B daripada ke Siswa A.

Jarak A ke B: $d_{SMC}(A, B)$

- A = (1, 0, 1)
- B = (1, 1, 1)
- Jumlah Cocok = 2
- Jumlah Tidak Cocok = 1
- Total Variabel = 3

$$SMC = 2/3$$

$$d_{SMC}(A, B) = 1/3 \approx \mathbf{0.333}$$

Jarak A ke C: $d_{SMC}(A, C)$

- A = (1, 0, 1)
- C = (0, 1, 1)
- Jumlah Cocok = 1
- Jumlah Tidak Cocok = 2
- Total Variabel = 3

$$SMC = 1/3$$

$$d_{SMC}(A, C) = 2/3 \approx \mathbf{0.667}$$

Jarak B ke C: $d_{SMC}(B, C)$

- B = (1, 1, 1)
- C = (0, 1, 1)
- Jumlah Cocok = 2
- Jumlah Tidak Cocok = 1
- Total Variabel = 3

$$SMC = 2/3$$

$$d_{SMC}(B, C) = 1/3 \approx \mathbf{0.333}$$

Jarak berbasis Rank (Ordinal)

- Pemberian Peringkat (Ranking): Setiap kategori dalam variabel ordinal diberi nilai numerik sesuai urutannya. Misalnya: Buruk=1, Cukup=2, Baik=3, Sangat Baik=4.
- Normalisasi Peringkat: Karena variabel yang berbeda mungkin memiliki jumlah tingkatan yang berbeda (misal, satu 3 tingkat, satu lagi 5 tingkat), peringkat perlu dinormalisasi agar sebanding. Rumus normalisasi yang umum adalah mengubah peringkat ke rentang [0, 1]:

$$z_{if} = \frac{r_{if} - 1}{M_f - 1}$$

- Setelah semua variabel ordinal dinormalisasi, bisa menghitung jarak antar objek menggunakan metrik seperti Jarak Manhattan.

Jarak berbasis Rank (Ordinal)

Pelanggan	Kepuasan Layanan (X_1)	Frekuensi Kunjungan (X_2)	Tingkat Rekomendasi (X_3)
A	Sangat Puas	Sering	Tinggi
B	Puas	Kadang-kadang	Sedang
C	Sangat Puas	Jarang	Rendah

Langkah 1: Tentukan Peringkat dan M_f

- X_1 (Kepuasan): Buruk(1), Cukup(2), Puas(3), Sangat Puas(4) $\rightarrow M_1 = 4$
- X_2 (Frekuensi): Jarang(1), Kadang-kadang(2), Sering(3) $\rightarrow M_2 = 3$
- X_3 (Rekomendasi): Rendah(1), Sedang(2), Tinggi(3) $\rightarrow M_3 = 3$

Langkah 2: Ubah Data ke Peringkat (r_{if})

- A = (4, 3, 3)
- B = (3, 2, 2)
- C = (4, 1, 1)

Langkah 3: Normalisasi Peringkat (z_{if})

- A:
 - $z_{A1} = (4 - 1)/(4 - 1) = 3/3 = 1.0$
 - $z_{A2} = (3 - 1)/(3 - 1) = 2/2 = 1.0$
 - $z_{A3} = (3 - 1)/(3 - 1) = 2/2 = 1.0$
 - $A' = (1.0, 1.0, 1.0)$
- B:
 - $z_{B1} = (3 - 1)/(4 - 1) = 2/3 \approx 0.67$
 - $z_{B2} = (2 - 1)/(3 - 1) = 1/2 = 0.5$
 - $z_{B3} = (2 - 1)/(3 - 1) = 1/2 = 0.5$
 - $B' = (0.67, 0.5, 0.5)$
- C:
 - $z_{C1} = (4 - 1)/(4 - 1) = 3/3 = 1.0$
 - $z_{C2} = (1 - 1)/(3 - 1) = 0/2 = 0.0$
 - $z_{C3} = (1 - 1)/(3 - 1) = 0/2 = 0.0$
 - $C' = (1.0, 0.0, 0.0)$

Jarak berbasis Rank (Ordinal)

Langkah 4: Hitung Jarak (menggunakan Jarak Manhattan)

- $d(A', B')$:

$$|1.0 - 0.67| + |1.0 - 0.5| + |1.0 - 0.5| \\ = 0.33 + 0.5 + 0.5 = \mathbf{1.33}$$

- $d(A', C')$:

$$|1.0 - 1.0| + |1.0 - 0.0| + |1.0 - 0.0| \\ = 0 + 1.0 + 1.0 = \mathbf{2.0}$$

- $d(B', C')$:

$$|0.67 - 1.0| + |0.5 - 0.0| + |0.5 - 0.0| \\ = |-0.33| + 0.5 + 0.5 = 0.33 + 0.5 + 0.5 = \mathbf{1.33}$$

Pelanggan A lebih dekat ke Pelanggan B

Pelanggan B memiliki kedekatan yang sama ke Pelanggan A dan Pelanggan C

Pelanggan C lebih dekat ke Pelanggan B

Berdasarkan Jarak Manhattan pada data ordinal yang dinormalisasi, A dan B sama dekatnya dengan B dan C. A dan C adalah yang paling jauh.

Jarak Gower

- Jarak Gower adalah metrik yang sangat berguna dan fleksibel karena dirancang khusus untuk menghitung jarak atau ketidakmiripan antara objek-objek yang memiliki campuran tipe data (numerik/kontinu, nominal, dan ordinal).
- Jarak Gower (d_{Gower}) antara dua objek (p dan q) dengan n variabel dihitung sebagai berikut:

$$d(p, q) = \frac{\sum_{f=1}^n w_f \cdot d_f(p_f, q_f)}{\sum_{f=1}^n w_f}$$

w_f : Bobot (weight) yang diberikan untuk variabel ke-f. Biasanya, w_f diatur ke 1 untuk semua variabel, artinya semua dianggap sama pentingnya. w_f bisa juga diatur ke 0 jika kita ingin mengabaikan variabel tertentu atau jika ada *missing value* pada perbandingan tersebut.

Jarak Gower

- **$d_f(p_f, q_f)$** : Jarak atau ketidakmiripan antara objek p dan q pada variabel ke- f . Perhitungannya berbeda tergantung tipe data:

- Untuk Data Nominal/Biner Simetris (X_f):

- $d_f = 0$ jika $p_f = q_f$ (cocok)
- $d_f = 1$ jika $p_f \neq q_f$ (tidak cocok)

- Untuk Data Ordinal (X_f):

1. Ubah kategori menjadi peringkat (r_{if}).
2. Normalisasi peringkat ke $[0, 1]$ menggunakan $z_{if} = \frac{r_{if}-1}{M_f-1}$, di mana M_f adalah jumlah peringkat.
3. Hitung jarak sebagai $d_f = |z_{pf} - z_{qf}|$.

Untuk Data Kontinu/Rasio (X_f):

$$d_f = \frac{|p_f - q_f|}{R_f}$$

Hasil Jarak Gower selalu berada di antara 0 (objek identik) dan 1 (objek sangat berbeda).

Di mana R_f adalah rentang (nilai maksimum - nilai minimum) dari variabel f di seluruh dataset. Ini secara efektif menormalkan jarak ke rentang $[0, 1]$.

Jarak Gower

Individu	Tinggi (cm) (X_1)	Pekerjaan (X_2)	Kepuasan (X_3)	Punya Mobil (X_4)
A	170	Karyawan	Puas	Ya
B	180	Wiraswasta	Cukup	Tidak
C	165	Karyawan	Sangat Puas	Ya

Langkah 1: Identifikasi Tipe Data dan Parameter

- X_1 : Kontinu. Rentang $R_1 = 180 - 165 = 15$.
- X_2 : Nominal.
- X_3 : Ordinal. Peringkat: Cukup(1), Puas(2), Sangat Puas(3). $M_3 = 3$.
- X_4 : Biner. Ya(1), Tidak(0).
- Kita asumsikan $w_f = 1$ untuk semua f , jadi $\sum w_f = 4$.

Langkah 2: Transformasi Data Ordinal

- A: Puas(2) $\rightarrow z = (2 - 1)/(3 - 1) = 1/2 = 0.5$
- B: Cukup(1) $\rightarrow z = (1 - 1)/(3 - 1) = 0/2 = 0.0$
- C: Sangat Puas(3) $\rightarrow z = (3 - 1)/(3 - 1) = 2/2 = 1.0$

Data Siap Hitung:

Individu	X_1	X_2	X_3 (norm)	X_4
A	170	Karyawan	0.5	1
B	180	Wiraswasta	0.0	0
C	165	Karyawan	1.0	1

Jarak Gower

Data Siap Hitung:

Individu	X_1	X_2	X_3 (norm)	X_4
A	170	Karyawan	0.5	1
B	180	Wiraswasta	0.0	0
C	165	Karyawan	1.0	1

A vs B

- d_1 (Tinggi): $\frac{|170-180|}{15} = \frac{10}{15} \approx 0.67$
- d_2 (Pekerjaan): Karyawan vs Wiraswasta $\implies$ Tidak Cocok $\implies 1.0$
- d_3 (Kepuasan): $|0.5 - 0.0| = 0.5$
- d_4 (Punya Mobil): 1 vs 0 $\implies$ Tidak Cocok $\implies 1.0$
- Jarak $d(A, B)$: $\frac{(1 \times 0.67) + (1 \times 1.0) + (1 \times 0.5) + (1 \times 1.0)}{4} = \frac{3.17}{4} \approx \mathbf{0.79}$

A vs C

- d_1 (Tinggi): $\frac{|170-165|}{15} = \frac{5}{15} \approx 0.33$
- d_2 (Pekerjaan): Karyawan vs Karyawan $\implies$ Cocok $\implies 0.0$
- d_3 (Kepuasan): $|0.5 - 1.0| = |-0.5| = 0.5$
- d_4 (Punya Mobil): 1 vs 1 $\implies$ Cocok $\implies 0.0$
- Jarak $d(A, C)$: $\frac{(1 \times 0.33) + (1 \times 0.0) + (1 \times 0.5) + (1 \times 0.0)}{4} = \frac{0.83}{4} \approx \mathbf{0.21}$

B vs C

- d_1 (Tinggi): $\frac{|180-165|}{15} = \frac{15}{15} = 1.0$
- d_2 (Pekerjaan): Wiraswasta vs Karyawan $\implies$ Tidak Cocok $\implies 1.0$
- d_3 (Kepuasan): $|0.0 - 1.0| = |-1.0| = 1.0$
- d_4 (Punya Mobil): 0 vs 1 $\implies$ Tidak Cocok $\implies 1.0$
- Jarak $d(B, C)$: $\frac{(1 \times 1.0) + (1 \times 1.0) + (1 \times 1.0) + (1 \times 1.0)}{4} = \frac{4.0}{4} = \mathbf{1.0}$

- Untuk A: Jarak A ke C (0.21) lebih kecil daripada jarak A ke B (0.79). Maka, A lebih dekat ke C.
- Untuk B: Jarak B ke A (0.79) lebih kecil daripada jarak B ke C (1.0). Maka, B lebih dekat ke A.
- Untuk C: Jarak C ke A (0.21) lebih kecil daripada jarak C ke B (1.0). Maka, C lebih dekat ke A.

Section 2

Linkage

Linkage

- Konsep linkage (atau keterkaitan) dalam analisis klastering, adalah aturan atau kriteria yang digunakan untuk mengukur jarak antara dua klaster.
- Ingat, dalam klastering, kita memulai dengan setiap objek sebagai klaster tunggal. Kemudian, pada setiap langkah, kita menggabungkan dua klaster yang "paling dekat". "Linkage" inilah yang mendefinisikan apa artinya "paling dekat" ketika kita berurusan dengan kelompok-kelompok titik (klaster), bukan hanya titik-titik individual.

Single Linkage (Keterkaitan Tunggal / Tetangga Terdekat)

- Konsep: Jarak antara dua kluster didefinisikan sebagai jarak minimum antara dua objek, di mana satu objek berasal dari kluster pertama dan objek lainnya dari kluster kedua.
- Cara Kerja: Mencari pasangan titik terdekat (satu dari masing-masing kluster) dan menggunakan jarak mereka sebagai jarak antar kluster. Kecenderungan:
- Cenderung menghasilkan kluster yang panjang seperti "rantai" dan dapat menggabungkan kluster yang relatif jauh jika ada satu "jembatan" titik yang dekat. Baik untuk menemukan bentuk kluster yang tidak elips, tetapi sangat sensitif terhadap noise (derau) dan outlier.

Single Linkage (Keterkaitan Tunggal / Tetangga Terdekat)

Misalkan kita memiliki 4 titik (objek) dalam ruang 2D:

- $A = (2, 2)$
- $B = (3, 2)$
- $C = (9, 8)$
- $D = (8, 9)$

$$D_0 = \begin{pmatrix} & A & B & C & D \\ A & 0 & 1.00 & 9.22 & 9.22 \\ B & 1.00 & 0 & 8.49 & 8.60 \\ C & 9.22 & 8.49 & 0 & 1.41 \\ D & 9.22 & 8.60 & 1.41 & 0 \end{pmatrix}$$

Pada tahap ini, jarak terkecil adalah $d(A, B) = 1.00$, sehingga A dan B digabungkan menjadi kluster (AB).

Langkah 0: Hitung Jarak Awal Antar Titik

Kita hitung jarak Euclidean antara semua pasangan titik:

- $d(A, B) = \sqrt{(3 - 2)^2 + (2 - 2)^2} = \sqrt{1^2 + 0^2} = \sqrt{1} = \mathbf{1.0}$
- $d(C, D) = \sqrt{(8 - 9)^2 + (9 - 8)^2} = \sqrt{(-1)^2 + 1^2} = \sqrt{2} \approx \mathbf{1.41}$
- $d(A, C) = \sqrt{(9 - 2)^2 + (8 - 2)^2} = \sqrt{7^2 + 6^2} = \sqrt{49 + 36} = \sqrt{85} \approx 9.22$
- $d(A, D) = \sqrt{(8 - 2)^2 + (9 - 2)^2} = \sqrt{6^2 + 7^2} = \sqrt{36 + 49} = \sqrt{85} \approx 9.22$
- $d(B, C) = \sqrt{(9 - 3)^2 + (8 - 2)^2} = \sqrt{6^2 + 6^2} = \sqrt{36 + 36} = \sqrt{72} \approx 8.49$
- $d(B, D) = \sqrt{(8 - 3)^2 + (9 - 2)^2} = \sqrt{5^2 + 7^2} = \sqrt{25 + 49} = \sqrt{74} \approx 8.60$

Single Linkage (Keterkaitan Tunggal / Tetangga Terdekat)

Langkah 1: Penggabungan Pertama

- Cari jarak terkecil di matriks. Jarak terkecil adalah 1.0 antara A dan B.
- Gabungkan A dan B menjadi klaster baru: (AB).
- Kita sekarang memiliki 3 klaster: (AB), C, D.

Langkah 2: Perbarui Jarak & Penggabungan Kedua

- Kita perlu menghitung jarak dari klaster baru (AB) ke klaster C dan D menggunakan *Single Linkage*.
- Jarak (AB) ke C:
 - $d((AB), C) = \min(d(A, C), d(B, C))$
 - $d((AB), C) = \min(9.22, 8.49) = \mathbf{8.49}$
- Jarak (AB) ke D:
 - $d((AB), D) = \min(d(A, D), d(B, D))$
 - $d((AB), D) = \min(9.22, 8.60) = \mathbf{8.60}$
- Jarak C ke D tetap 1.41.

$$D_1 = \begin{pmatrix} & (AB) & C & D \\ (AB) & 0 & 8.49 & 8.60 \\ C & 8.49 & 0 & 1.41 \\ D & 8.60 & 1.41 & 0 \end{pmatrix}$$

Pada tahap ini, jarak terkecil adalah $d(C, D) = 1.41$, sehingga C dan D digabungkan menjadi klaster (CD).

Single Linkage (Keterkaitan Tunggal / Tetangga Terdekat)

Langkah 3: Perbarui Jarak & Penggabungan Ketiga (Terakhir)

- Kita hitung jarak antara (AB) dan (CD) menggunakan *Single Linkage*.
- Jarak (AB) ke (CD):
 - $d((AB), (CD)) = \min(d(A, C), d(A, D), d(B, C), d(B, D))$
 - $d((AB), (CD)) = \min(9.22, 9.22, 8.49, 8.60) = \mathbf{8.49}$
- Gabungkan (AB) dan (CD) menjadi kluster (ABCD). Jarak penggabungan terakhir adalah 8.49.

$$D_2 = \begin{pmatrix} & (AB) & (CD) \\ (AB) & 0 & 8.49 \\ (CD) & 8.49 & 0 \end{pmatrix}$$

Pada tahap ini, satu-satunya jarak adalah 8.49, sehingga (AB) dan (CD) digabungkan.

1. A dan B bergabung pada jarak 1.0.
2. C dan D bergabung pada jarak 1.41.
3. Kluster (AB) dan (CD) bergabung pada jarak 8.49.

Complete Linkage (Keterkaitan Lengkap / Tetangga Terjauh)

Langkah 1: Penggabungan Pertama

- Sama seperti *Single Linkage*, kita mencari jarak terkecil di matriks awal. Jarak terkecil adalah 1.00 antara A dan B.
- Gabungkan A dan B menjadi kluster baru: (AB).
- Kita sekarang memiliki 3 kluster: (AB), C, D.

Langkah 2: Perbarui Jarak & Penggabungan Kedua

- Kita perlu menghitung jarak dari kluster baru (AB) ke kluster C dan D menggunakan *Complete Linkage*.
- Jarak (AB) ke C:
 - $d((AB), C) = \max(d(A, C), d(B, C))$ <- Perhatikan: kita pakai `max` sekarang!
 - $d((AB), C) = \max(9.22, 8.49) = \mathbf{9.22}$
- Jarak (AB) ke D:
 - $d((AB), D) = \max(d(A, D), d(B, D))$
 - $d((AB), D) = \max(9.22, 8.60) = \mathbf{9.22}$
- Jarak C ke D tetap 1.41.

$$D_1 = \begin{pmatrix} & (AB) & C & D \\ (AB) & 0 & 9.22 & 9.22 \\ C & 9.22 & 0 & \mathbf{1.41} \\ D & 9.22 & 1.41 & 0 \end{pmatrix}$$

Jarak terkecil adalah 1.41 antara C dan D

Complete Linkage (Keterkaitan Lengkap / Tetangga Terjauh)

Langkah 3: Perbarui Jarak & Penggabungan Ketiga (Terakhir)

- Kita hitung jarak antara (AB) dan (CD) menggunakan *Complete Linkage*.
- Jarak (AB) ke (CD):
 - $d((AB), (CD)) = \max(d(A, C), d(A, D), d(B, C), d(B, D))$ <- Cari jarak terjauh!
 - $d((AB), (CD)) = \max(9.22, 9.22, 8.49, 8.60) = \mathbf{9.22}$
- Gabungkan (AB) dan (CD) menjadi kluster (ABCD). Jarak penggabungan terakhir adalah 9.22.

Proses ini menghasilkan urutan penggabungan yang sama dengan *Single Linkage* untuk contoh ini (A-B, lalu C-D, lalu (AB)-(CD)), TETAPI perhatikan jarak penggabungan terakhirnya:

- *Single Linkage*: (AB) dan (CD) bergabung pada jarak 8.49 (jarak terdekat antar kluster).
- *Complete Linkage*: (AB) dan (CD) bergabung pada jarak 9.22 (jarak terjauh antar kluster).

Average Linkage (Keterkaitan Rata-rata / UPGMA)

Langkah 1: Penggabungan Pertama

- Sama seperti sebelumnya, kita cari jarak terkecil, yaitu 1.00 antara A dan B.
- Gabungkan A dan B menjadi kluster baru: (AB).
- Kita sekarang memiliki 3 kluster: (AB), C, D.

Langkah 2: Perbarui Jarak & Penggabungan Kedua

- Kita perlu menghitung jarak dari kluster baru (AB) ke kluster C dan D menggunakan *Average Linkage*.
- Jarak (AB) ke C:
 - Kita hitung rata-rata dari semua pasangan antara (AB) dan C. Ada 2 pasangan: (A, C) dan (B, C).
 - $d((AB), C) = \frac{d(A,C) + d(B,C)}{2 \times 1}$
 - $d((AB), C) = \frac{9.22 + 8.49}{2} = \frac{17.71}{2} = \mathbf{8.855}$

Average Linkage (Keterkaitan Rata-rata / UPGMA)

- Jarak (AB) ke D:
 - Kita hitung rata-rata dari semua pasangan antara (AB) dan D. Ada 2 pasangan: (A, D) dan (B, D).
 - $d((AB), D) = \frac{d(A,D)+d(B,D)}{2 \times 1}$
 - $d((AB), D) = \frac{9.22+8.60}{2} = \frac{17.82}{2} = \mathbf{8.91}$
- Jarak C ke D tetap 1.41.

Matriks Jarak Baru:

$$D_1 = \begin{pmatrix} & (AB) & C & D \\ (AB) & 0 & 8.855 & 8.91 \\ C & 8.855 & 0 & \mathbf{1.41} \\ D & 8.91 & 1.41 & 0 \end{pmatrix}$$

- Cari jarak terkecil di matriks baru. Jarak terkecil adalah **1.41** antara C dan D.
- Gabungkan C dan D menjadi kluster baru: **(CD)**.
- Kita sekarang memiliki 2 kluster: (AB), (CD).

Average Linkage (Keterkaitan Rata-rata / UPGMA)

Langkah 3: Perbarui Jarak & Penggabungan Ketiga (Terakhir)

- Kita hitung jarak antara (AB) dan (CD) menggunakan *Average Linkage*.
- Kita hitung rata-rata dari *semua* pasangan antara (AB) dan (CD). Ada $2 \times 2 = 4$ pasangan: (A, C), (A, D), (B, C), dan (B, D).
- Jarak (AB) ke (CD):
 - $d((AB), (CD)) = \frac{d(A,C) + d(A,D) + d(B,C) + d(B,D)}{2 \times 2}$
 - $d((AB), (CD)) = \frac{9.22 + 9.22 + 8.49 + 8.60}{4}$
 - $d((AB), (CD)) = \frac{35.53}{4} = \mathbf{8.8825}$
- Gabungkan (AB) dan (CD) menjadi kluster (ABCD). Jarak penggabungan terakhir adalah 8.8825.

Struktur penggabungan masih sama, tetapi jarak penggabungan terakhir berbeda lagi:

- Single Linkage: 8.49
- Complete Linkage: 9.22
- Average Linkage: 8.8825

Centroid Linkage (Keterkaitan Sentroid)

Langkah 1: Penggabungan Pertama

- Jarak terkecil adalah 1.00 antara A dan B.
- Gabungkan A dan B menjadi kluster (AB).
- Hitung Sentroid (AB):
 - $C_{AB} = \left(\frac{2+3}{2}, \frac{2+2}{2}\right) = (2.5, 2)$
- Kita sekarang memiliki 3 kluster: (AB), C, D. Sentroid mereka adalah (2.5, 2), (9, 8), dan (8, 9).

Langkah 2: Perbarui Jarak & Penggabungan Kedua

- Kita perlu menghitung jarak antar sentroid menggunakan *Centroid Linkage*.
- Jarak (AB) ke C:
 - $d((AB), C) = d(C_{AB}, C_C) = \sqrt{(9 - 2.5)^2 + (8 - 2)^2}$
 - $d((AB), C) = \sqrt{6.5^2 + 6^2} = \sqrt{42.25 + 36} = \sqrt{78.25} \approx 8.846$
- Jarak (AB) ke D:
 - $d((AB), D) = d(C_{AB}, C_D) = \sqrt{(8 - 2.5)^2 + (9 - 2)^2}$
 - $d((AB), D) = \sqrt{5.5^2 + 7^2} = \sqrt{30.25 + 49} = \sqrt{79.25} \approx 8.902$
- Jarak C ke D tetap 1.41 (karena C dan D masih titik tunggal, jarak sentroidnya adalah jarak antar titik).

Centroid Linkage (Keterkaitan Sentroid)

Matriks Jarak Baru:

$$D_1 = \begin{pmatrix} & (AB) & C & D \\ (AB) & 0 & 8.846 & 8.902 \\ C & 8.846 & 0 & \mathbf{1.41} \\ D & 8.902 & 1.41 & 0 \end{pmatrix}$$

- Cari jarak terkecil di matriks baru. Jarak terkecil adalah 1.41 antara C dan D.
- Gabungkan C dan D menjadi kluster (CD).
- Hitung Sentroid (CD):
 - $C_{CD} = \left(\frac{9+8}{2}, \frac{8+9}{2}\right) = (8.5, 8.5)$
- Kita sekarang memiliki 2 kluster: (AB) dan (CD). Sentroid mereka adalah (2.5, 2) dan (8.5, 8.5).

Langkah 3: Perbarui Jarak & Penggabungan Ketiga (Terakhir)

- Kita hitung jarak antara (AB) dan (CD) menggunakan *Centroid Linkage*.
- Jarak (AB) ke (CD):
 - $d((AB), (CD)) = d(C_{AB}, C_{CD}) = \sqrt{(8.5 - 2.5)^2 + (8.5 - 2)^2}$
 - $d((AB), (CD)) = \sqrt{6^2 + 6.5^2} = \sqrt{36 + 42.25} = \sqrt{78.25} \approx \mathbf{8.846}$
- Gabungkan (AB) dan (CD) menjadi kluster (ABCD). Jarak penggabungan terakhir adalah 8.846.

Section 3

Hierarki – Clustering (Agglomerative)

Hierarki – Clustering (Divisive)

Hierarki Cluster

Pengklasteraan hierarki adalah teknik dalam analisis data yang bertujuan untuk membangun hierarki klaster dalam bentuk struktur pohon (dendrogram). Berbeda dengan metode partisi (seperti K-Means), tidak perlu menentukan jumlah klaster di awal. Sebaliknya, mendapatkan representasi visual yang menunjukkan bagaimana objek-objek data dapat dikelompokkan pada berbagai tingkat kesamaan.

Hierarki Cluster - Kelebihan

1. Tidak Perlu Menentukan Jumlah Klaster (K) di Awal: Ini adalah salah satu keunggulan terbesar. Anda tidak perlu menebak berapa banyak klaster yang ada; Anda bisa menentukannya nanti setelah melihat struktur pohon (dendrogram).
2. Visualisasi (Dendrogram): Hasilnya berupa dendrogram yang memberikan visualisasi intuitif tentang bagaimana objek-objek data saling berhubungan pada berbagai tingkat kesamaan. Ini membantu memahami struktur data secara mendalam.
3. Struktur Bertingkat: Memberikan informasi lebih kaya daripada sekadar partisi datar. Anda bisa melihat hubungan sub-klaster di dalam klaster yang lebih besar.
4. Fleksibilitas Metrik Jarak dan Linkage: Anda bisa memilih berbagai kombinasi metrik jarak dan metode linkage yang paling sesuai dengan karakteristik data dan tujuan analisis Anda.

Hierarki Cluster - Kekurangan

1. Komputasi Mahal: Terutama untuk dataset besar (N besar), perhitungan matriks jarak ($N \times N$) dan pembaruannya pada setiap langkah bisa menjadi sangat lambat dan memakan banyak memori.
2. Keputusan Penggabungan/Pembagian Bersifat Final: Sekali objek digabungkan (dalam aglomeratif) atau dipisahkan (dalam divisif), keputusan tersebut tidak dapat diubah pada langkah-langkah berikutnya. Ini bisa menyebabkan hasil yang kurang optimal jika ada penggabungan yang "salah" di awal.
3. Tidak Ada Koreksi Kesalahan: Berbeda dengan metode iteratif (seperti K-Means) yang bisa memperbaiki penempatan objek, metode hierarki tidak memiliki mekanisme koreksi seperti itu.
4. Sulit Ditentukan Jumlah Kluster Optimal: Meskipun tidak perlu menentukan K di awal, memilih "potongan" yang tepat pada dendrogram untuk menentukan jumlah kluster akhir bisa bersifat subjektif dan memerlukan metode validasi tambahan.
5. Sensitivitas Terhadap Noise dan Outlier: Beberapa metode linkage (terutama Single Linkage) bisa sangat sensitif terhadap noise dan outlier.
6. Kurang Praktis untuk Data Sangat Besar: Karena kompleksitas $O(N^2)$ atau bahkan $O(N^3)$, metode hierarki seringkali tidak praktis untuk dataset dengan puluhan atau ratusan ribu objek.

Hierarki Cluster

- Klastering Aglomeratif (Agglomerative Clustering)
 - Klastering Divisif (Divisive Clustering)
-
- Output : Dendogram

Klastering Aglomeratif (Agglomerative Clustering)

- Ini adalah pendekatan yang paling sering digunakan.
- Prosesnya adalah sebagai berikut:
 1. Inisialisasi: Setiap objek (titik data) dimulai sebagai klasternya sendiri (jadi, jika Anda punya N objek, Anda mulai dengan N klaster).
 2. Hitung Jarak: Hitung matriks jarak (menggunakan metrik seperti Euclidean, Manhattan, Gower, dll.) antara semua pasangan objek/klaster.
 3. Gabung (Merge): Temukan dua klaster yang paling dekat (paling mirip) berdasarkan matriks jarak. Gabungkan kedua klaster ini menjadi satu klaster baru.
 4. Perbarui Jarak: Hitung ulang matriks jarak, sekarang dengan menyertakan klaster baru. Di sinilah metode linkage berperan untuk menentukan jarak antara klaster baru ini dengan klaster lainnya.
 5. Ulangi: Ulangi langkah 3 dan 4 sampai semua objek tergabung dalam satu klaster besar.

Klastering Aglomeratif (Agglomerative Clustering)

Objek	X1	X2	X3
O1	1	2	1
O2	2	1	1
O3	5	6	5
O4	6	5	5
O5	10	10	10

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 0: Matriks Jarak Awal (D_0)

Hitung semua kombinasi jarak Euclidean:

$$d = \sqrt{(X_{i1} - X_{j1})^2 + (X_{i2} - X_{j2})^2 + (X_{i3} - X_{j3})^2}$$

$$d(1,2) = \sqrt{(1 - 2)^2 + (2 - 1)^2 + (1 - 1)^2} = \sqrt{1 + 1 + 0} = 1.41$$

$$d(3,4) = \sqrt{(5 - 6)^2 + (6 - 5)^2 + (5 - 5)^2} = \sqrt{1 + 1 + 0} = 1.41$$

$$d(1,3) = \sqrt{(1 - 5)^2 + (2 - 6)^2 + (1 - 5)^2} = \sqrt{16 + 16 + 16} = 6.93$$

$$d(3,5) = \sqrt{(5 - 10)^2 + (6 - 10)^2 + (5 - 10)^2} = \sqrt{25 + 16 + 25} = 8.12$$

Lakukan untuk semua Kombinasi Observasi.

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 0: Matriks Jarak Awal (D_0)

	O1	O2	O3	O4	O5
O1	0	1.41	6.93	7.07	15.03
O2	1.41	0	7.07	6.93	15.03
O3	6.93	7.07	0	1.41	8.12
O4	7.07	6.93	1.41	0	8.12
O5	15.03	15.03	8.12	8.12	0

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 1: Penggabungan O1 dan O2

Jarak terkecil adalah 1.41. Pilih misal O1 dan O2 untuk digabung menjadi kluster (1,2).

Perhitungan Jarak Baru (Single Linkage):

- $d((1,2), 3) = \min(d(1,3), d(2,3)) = \min(6.93, 7.07) = 6.93$
- $d((1,2), 4) = \min(d(1,4), d(2,4)) = \min(7.07, 6.93) = 6.93$
- $d((1,2), 5) = \min(d(1,5), d(2,5)) = \min(15.03, 15.03) = 15.03$

Matriks Jarak (D_1)

	(1,2)	O3	O4	O5
(1,2)	0	6.93	6.93	15.03
O3	6.93	0	1.41	8.12
O4	6.93	1.41	0	8.12
O5	15.03	8.12	8.12	0

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 2: Penggabungan O3 dan O4

Jarak terkecil berikutnya adalah 1.41 antara O3 dan O4. Gabung menjadi klaster (3,4).

Perhitungan Jarak Baru:

- $d((1,2), (3,4)) = \min(d((1,2), 3), d((1,2), 4)) = \min(6.93, 6.93) = 6.93$
- $d((3,4), 5) = \min(d(3,5), d(4,5)) = \min(8.12, 8.12) = 8.12$

Matriks Jarak (D_2)

	(1,2)	(3,4)	O5
(1,2)	0	6.93	15.03
(3,4)	6.93	0	8.12
O5	15.03	8.12	0

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 3: Penggabungan (1,2) dan (3,4)

Jarak terkecil adalah 6.93. Maka klaster (1,2) bergabung dengan klaster (3,4) menjadi klaster (1,2,3,4).

Perhitungan Jarak Terakhir:

$$\bullet d((1,2,3,4), 5) = \min(d((1,2), 5), d((3,4), 5)) = \min(15.03, 8.12) = 8.12$$

Matriks Jarak (D_3)

	(1,2,3,4)	O5
(1,2,3,4)	0	8.12
O5	8.12	0

Klastering Aglomeratif (Agglomerative Clustering)

Iterasi 4: Penggabungan Akhir

Terakhir, objek O5 bergabung dengan klaster besar di ketinggian 8.12. Semua objek sekarang berada dalam satu klaster tunggal.

Dendrogram (Tinggi Gabung):

- 1.O1 & O2: Tinggi 1.41
- 2.O3 & O4: Tinggi 1.41
- 3.(1,2) & (3,4): Tinggi 6.93
- 4.(1,2,3,4) & O5: Tinggi 8.12

Klastering Aglomeratif (Agglomerative Clustering)

1. Persiapan

```
Datadf <- data.frame( x1 = c(1, 2, 5, 6, 10), x2 =  
c(2, 1, 6, 5, 10), x3 = c(1, 1, 5, 5, 10))  
rownames(df) <- paste0("O", 1:5)
```

2. Matriks Jarak (D0)

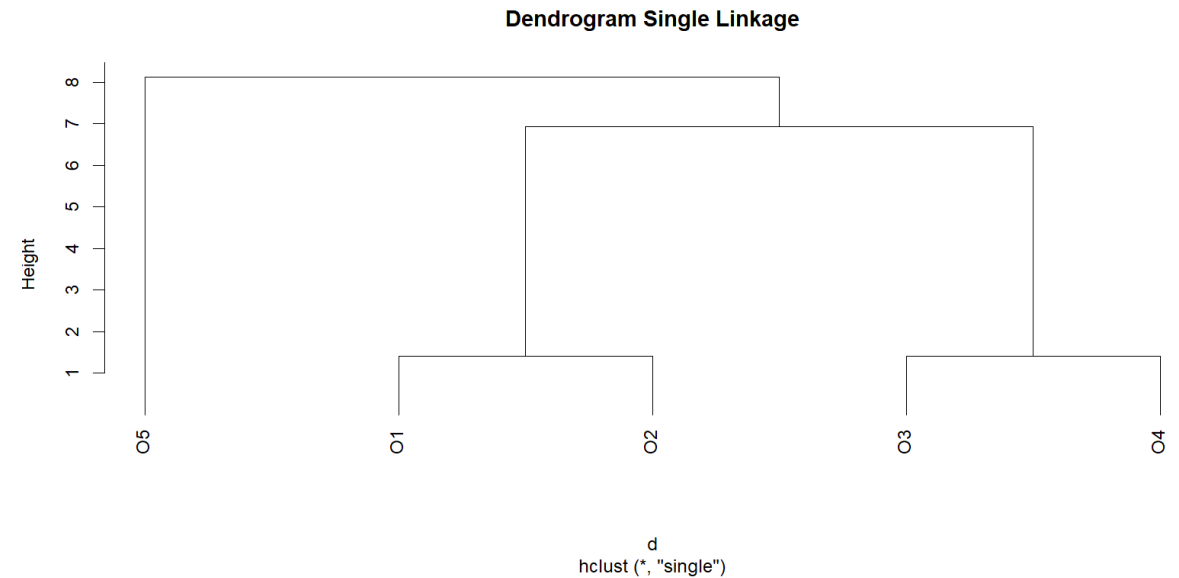
```
d <- dist(df, method = "euclidean")  
print(d)
```

3. Clustering & Dendrogram

```
hc <- hclust(d, method = "single")  
print(hc$height)
```

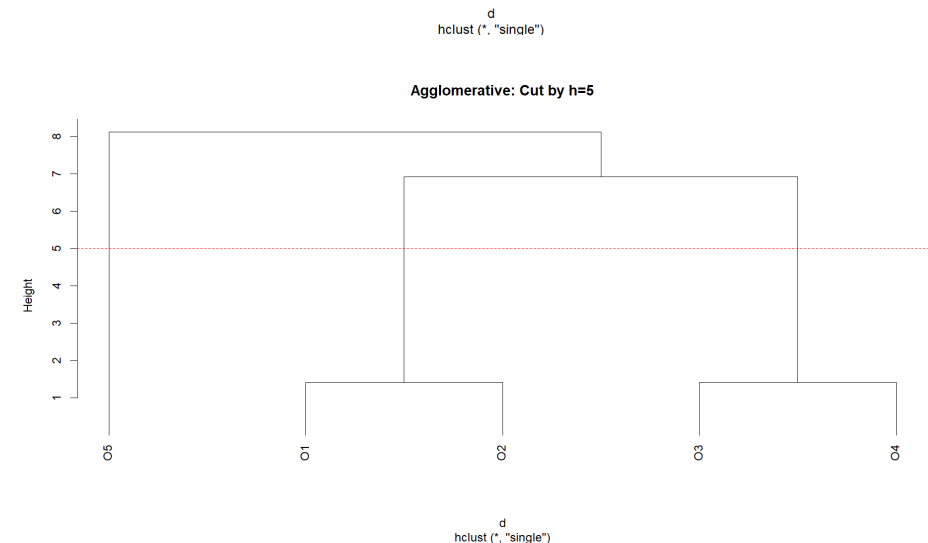
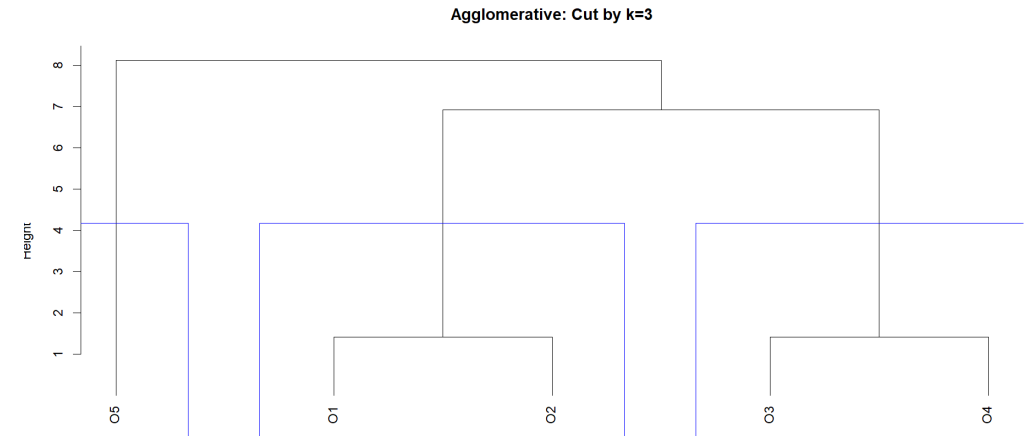
Cek tinggi penggabungan: 1.41, 1.41, 6.93, 8.12

```
plot(hc, hang = -1, main = "Dendrogram Single  
Linkage")
```



Klastering Aglomeratif (Agglomerative Clustering) – Cut Dendrogram

```
# A. Berdasarkan Jumlah Kluster (k = 3)
klaster_k <- cutree(hc, k = 3)
print(klaster_k)
# B. Berdasarkan Ketinggian (h = 5)
klaster_h <- cutree(hc, h = 5)
print(klaster_h)
# Visualisasi
plot(hc, hang = -1, main = "Agglomerative: Cut
by k=3")
rect.hclust(hc, k = 3, border = "blue")
# Garis biru untuk k=3
plot(hc, hang = -1, main = "Agglomerative: Cut
by h=5")
abline(h = 5, col = "red", lty = 2)
# Garis referensi h=5
```



Klastering Divisif (Divisive Clustering)

Pendekatan ini bekerja secara berlawanan:

1. Inisialisasi: Semua objek dimulai sebagai satu kluster besar tunggal.
2. Bagi (Split): Pada setiap langkah, kluster yang ada dipilih dan dibagi menjadi dua sub-kluster yang paling "berbeda" atau "heterogen".
3. Ulangi: Proses pembagian ini diulangi sampai setiap objek berada dalam klasternya sendiri, atau sampai kriteria penghentian tertentu terpenuhi (misalnya, jumlah kluster yang diinginkan).

Klastering Divisif (Divisive Clustering)

Klastering divisif secara konseptual lebih kompleks karena pada setiap langkah, ada sangat banyak cara untuk membagi sebuah klaster.

Beberapa pendekatan meliputi:

1. Berbasis Keterpisahan: Mencari cara membagi klaster sehingga jarak antar dua sub-klaster baru menjadi maksimal.
2. Berbasis Kohesi: Mencari cara membagi sehingga dalam setiap sub-klaster baru, objek-objeknya se-homogen mungkin.
3. Algoritma DIANA (Divisive ANALysis): Salah satu algoritma divisif yang lebih dikenal. Ia bekerja dengan mencari objek dalam sebuah klaster yang paling tidak mirip dengan objek lain, lalu memulai sub-klaster baru dengannya dan memindahkan objek lain yang lebih dekat ke sub-klaster baru tersebut.

Klastering Divisif (Divisive Clustering)-DIANA

Objek	X1	X2	X3
O1	1	2	1
O2	2	1	2
O3	8	9	8
O4	9	8	9
O5	18	19	18

Klastering Divisif (Divisive Clustering)-DIANA

Rumus Dasar Perhitungan

- Jarak Euclidean antara objek i dan j :

$$d(i, j) = \sqrt{\sum_{k=1}^p (X_{ik} - X_{jk})^2}$$

- Rata-rata Disimilaritas Objek i terhadap Klaster $\mathcal{C}$:

$$d_{avg}(i, \mathcal{C}) = \frac{1}{|\mathcal{C}| - 1} \sum_{j \in \mathcal{C}, j \neq i} d(i, j)$$

Klastering Divisif (Divisive Clustering)-DIANA

Iterasi 1: Memecah Klaster Utama {O1, O2, O3, O4, O5}

Langkah A: Perhitungan Jarak Euclidean (Contoh Object 1)

- $d(1,2) = \sqrt{(1-2)^2 + (2-1)^2 + (1-2)^2} = \sqrt{1+1+1} = 1.732$
- $d(1,3) = \sqrt{(1-8)^2 + (2-9)^2 + (1-8)^2} = \sqrt{49+49+49} = 12.124$
- $d(1,4) = \sqrt{(1-9)^2 + (2-8)^2 + (1-9)^2} = \sqrt{64+36+64} = 12.806$
- $d(1,5) = \sqrt{(1-18)^2 + (2-19)^2 + (1-18)^2} = \sqrt{289+289+289} = 29.445$
- Lakukan untuk semua kombinasi

Matriks Jarak Awal (D_0) dan Rata-rata (d_{avg}):

	O1	O2	O3	O4	O5	davg
O1	0	1.732	12.124	12.806	29.445	14.027
O2	1.732	0	11.662	12.124	28.914	13.608
O3	12.124	11.662	0	1.732	17.321	10.710
O4	12.806	12.124	1.732	0	16.823	10.871
O5	29.445	28.914	17.321	16.823	0	23.126

Keputusan Split 1:

1. Nilai d_{avg} terbesar adalah O5 (23.126).
2. O5 memisahkan diri menjadi *Splinter Group*.
3. Hasil: Klaster A {O1, O2, O3, O4} dan Klaster B {O5}.

Klastering Divisif (Divisive Clustering)-DIANA

Iterasi 2: Memecah Klaster {O1, O2, O3, O4}

Langkah A: Matriks Jarak Klaster Sisa (D_1)

	O1	O2	O3	O4	davg
O1	0	1.732	12.124	12.806	8.887
O2	1.732	0	11.662	12.124	8.506
O3	12.124	11.662	0	1.732	8.506
O4	12.806	12.124	1.732	0	8.887

Langkah B: Evaluasi Splinter Group

1. Pilih O1 (d_{avg} terbesar) sebagai inisiator pembelahan.

2. Uji apakah O2 harus ikut O1:

1. $d(O2, O1) = 1.732$

2. Rata-rata jarak O2 ke grup {O3, O4} = $(11.662 + 12.124)/2 = \mathbf{11.893}$

3. Karena $1.732 < 11.893$, maka O2 ikut bergabung dengan O1.

3. Uji apakah O3 harus ikut {O1, O2}:

1. Rata-rata jarak O3 ke {O1, O2} = $(12.124 + 11.662) / 2 = 11.893$

2. Jarak O3 ke O4 = 1.732. Karena $11.893 > 1.732$, O3 tetap bersama O4.

Hasil Split 2: Klaster {O1, O2} dan {O3, O4}.

Klastering Divisif (Divisive Clustering)-DIANA

Iterasi 3 & 4: Singleton Splits

	O1	O2	O3	O4	davg
O1	0	1.732	12.124	12.806	8.887
O2	1.732	0	11.662	12.124	8.506
O3	12.124	11.662	0	1.732	8.506
O4	12.806	12.124	1.732	0	8.887

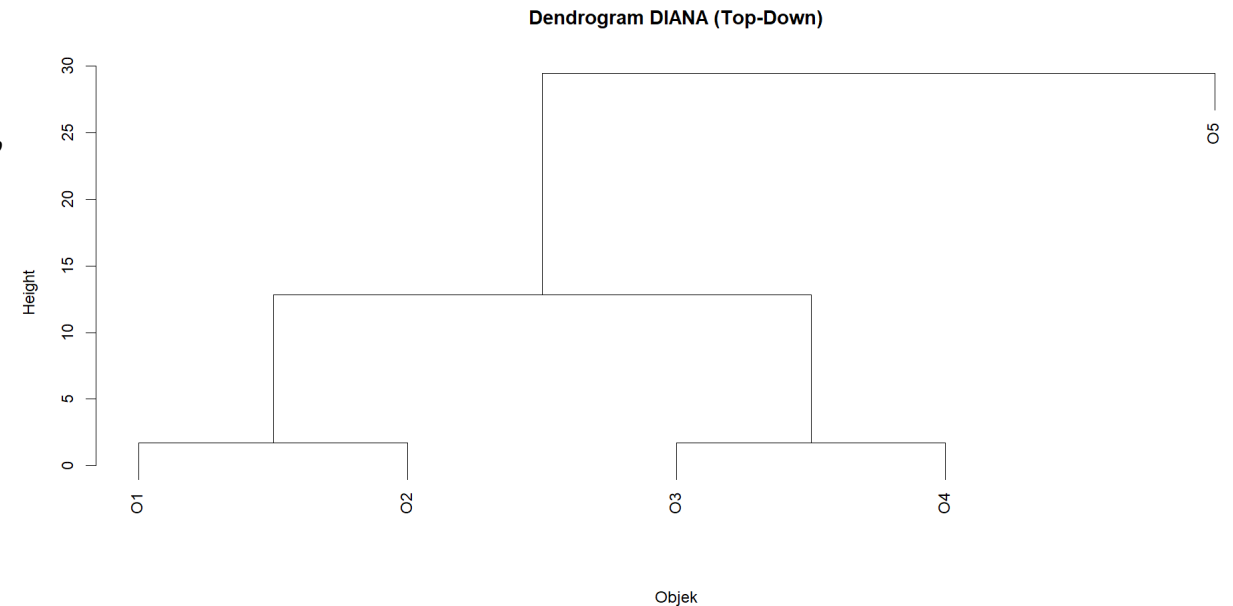
Split 3 (Klaster {O1, O2}):
Jarak internal $d(O1, O2) = 1.732$.
Klaster pecah menjadi individu: {O1} dan {O2}.

Split 4 (Klaster {O3, O4}):
Jarak internal $d(O3, O4) = 1.732$.
Klaster pecah menjadi individu: {O3} dan {O4}.

Iterasi	Klaster yang Terpecah	Hasil Pecahan	Kriteria Split	Tinggi di R (Diameter)
1	{O1, O2, O3, O4, O5}	{O1, O2, O3, O4} {O5}	23.126	29.445 (Jarak Terjauh)
2	{O1, O2, O3, O4}	{O1, O2} {O3, O4}	11.893	12.806 (Jarak Terjauh)
3	{O1, O2}	{O1} {O2}	1.732	1.732
4	{O3, O4}	{O3} {O4}	1.732	1.732

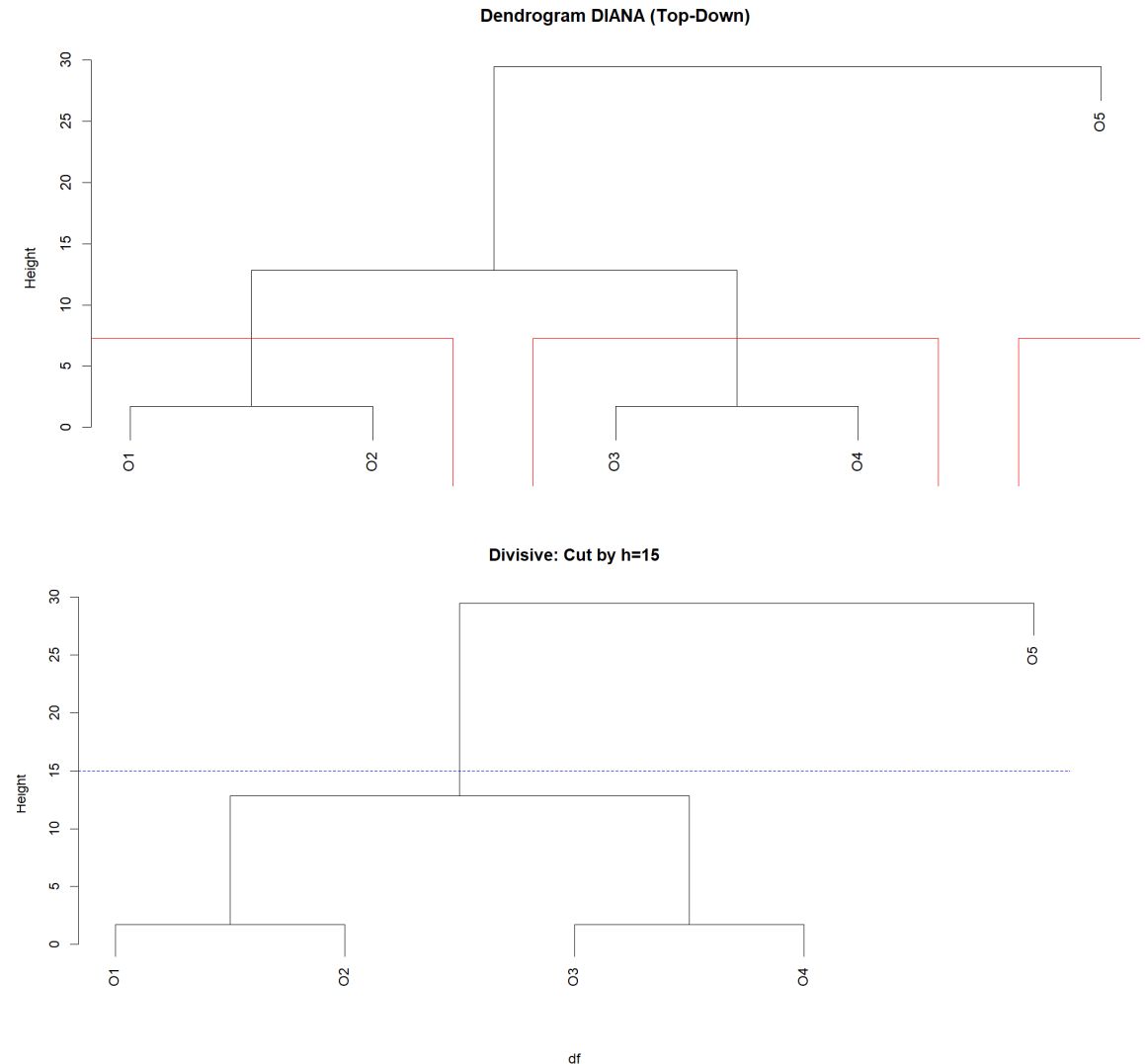
Klastering Divisif (Divisive Clustering)-DIANA

```
#DIANA
library(cluster)
# Input Data
df <- data.frame( x1 = c(1, 2, 8, 9, 18), x2 = c(2, 1, 9, 8, 19), x3 = c(1, 2, 8, 9, 18))
rownames(df) <- paste0("O", 1:5)
# Analisis Divisif
res_dc <- diana(df, metric = "euclidean")
# Tampilkan Matriks Jarak
print(as.matrix(dist(df)))
# Tampilkan Urutan Ketinggian
print(res_dc$height)
# Plot Dendrogram
plot(res_dc, which.plots = 2, main = "Dendrogram
DIANA (Top-Down)", xlab = "Objek", sub = "")
```



Klastering Divisif (Divisive Clustering)-DIANA

```
# A. Berdasarkan Jumlah Klaster (k = 3)
cut_div_k <- cutree(hc_div, k = 3)
print(cut_div_k)
# B. Berdasarkan Ketinggian (h = 15)
# Ingat: h pada DIANA adalah diameter kluster
cut_div_h <- cutree(hc_div, h = 15)
print(cut_div_h)
# Visualisasi plot(res_dc, which.plots = 2, main =
"Divisive: Cut by k=3")
rect.hclust(hc_div, k = 3, border = "red")
# Kotak merah untuk k=3
plot(res_dc, which.plots = 2, main = "Divisive: Cut by
h=15")
abline(h = 15, col = "blue", lty = 2)
# Garis referensi h=15
```



Section 4

Non Hierarki – Clustering (KMeans)

Non Hierarki – Clustering (KMedoid)

Non-Hierarki Cluster

- Jumlah Klaster (K) Ditentukan di Awal: Ini adalah ciri khas utama. Anda sebagai analis harus memutuskan berapa banyak klaster yang ingin Anda bentuk sebelum menjalankan algoritma. Menentukan nilai K yang optimal seringkali menjadi tantangan tersendiri.
- Partisi Tunggal: Hasil akhirnya adalah satu set pembagian data ke dalam K klaster. Anda tidak mendapatkan struktur pohon yang menunjukkan hubungan antar klaster pada level yang berbeda.
- Proses Iteratif: Sebagian besar metode non-hierarki bekerja melalui proses iteratif (pengulangan) untuk memperbaiki penempatan objek ke dalam klaster.
- Tujuan Optimasi: Biasanya, metode ini mencoba mengoptimalkan suatu fungsi objektif, seperti meminimalkan varians (atau jarak) dalam klaster dan memaksimalkan jarak antar klaster.
- Pusat Klaster (Centroid/Medoid): Banyak metode menggunakan konsep "pusat" klaster. Objek-objek kemudian ditugaskan ke klaster yang pusatnya paling dekat.

Non-Hierarki Cluster

Kelebihan:

- Secara komputasi umumnya lebih cepat daripada metode hierarki, terutama untuk dataset besar.
- Lebih mudah diinterpretasikan karena menghasilkan satu set kluster yang jelas.

Kekurangan:

- Harus menentukan K di awal, yang seringkali tidak diketahui.
- Hasilnya bisa sensitif terhadap pemilihan pusat kluster awal. Menjalankan algoritma beberapa kali dengan inisialisasi yang berbeda sering disarankan.
- Banyak metode (seperti K-Means) cenderung menghasilkan kluster yang berbentuk bulat (sferis) dan berukuran serupa, dan mungkin kesulitan menemukan kluster dengan bentuk lain.
- Bisa sensitif terhadap outlier (kecuali metode seperti K-Medoids).

Non-Hierarki Cluster

- K-Means
- K-Medoids
- DBSCAN
- CLARA

K-Means Algoritma

K-Means Clustering adalah salah satu algoritma pembelajaran mesin tanpa pengawasan (*unsupervised learning*) yang paling populer untuk mengelompokkan data. Berbeda dengan metode hierarki yang membentuk pohon (dendrogram), K-Means mempartisi data langsung ke dalam sejumlah **k** kluster yang telah ditentukan sebelumnya.

Tujuan dari algoritma ini adalah untuk meminimalkan variansi di dalam satu kluster (*intra-cluster variance*) dan memaksimalkan perbedaan antar kluster. Secara matematis, algoritma ini meminimalkan jumlah kuadrat jarak antara objek dengan pusat klasternya (*centroid*), yang dikenal sebagai **Within-Cluster Sum of Squares (WCSS)** atau **Sum of Squared Errors (SSE)**.

$$SSE = \sum_{j=1}^k \sum_{i \in C_j} \|x_i - \mu_j\|^2$$

Di mana:

- k = jumlah kluster.
- C_j = himpunan objek dalam kluster ke-j.
- x_i = objek data ke-i.
- μ_j = *centroid* (rata-rata) dari kluster ke-j.

K-Means Algoritma

Elbow Method adalah prosedur visual untuk menentukan jumlah kluster (k) optimal dengan cara membandingkan nilai SSE pada berbagai nilai k .

Prinsip Kerja:

Secara alami, nilai SSE akan terus menurun seiring bertambahnya jumlah kluster (k). Namun, penambahan k yang terlalu banyak akan membuat model menjadi tidak efisien (*overfitting*).

Logika Pengambilan Keputusan:

- Plot nilai k pada sumbu horizontal (X) dan nilai SSE pada sumbu vertikal (Y).
- Cari titik di mana terjadi penurunan SSE yang sangat tajam, diikuti dengan penurunan yang mulai melandai secara drastis pada titik berikutnya.
- Titik "patahan" tersebut menyerupai bentuk siku tangan (elbow). Titik inilah yang dipilih sebagai jumlah kluster optimal.

K-Means Algoritma

1. Inisialisasi (Initialization):

- Pilih K: Tentukan berapa banyak kluster (K) yang ingin Anda bentuk.
- Tentukan Sentroid Awal: Pilih K objek data secara acak dari dataset Anda untuk menjadi pusat kluster (sentroid) awal.
- Alternatif: Bisa juga menggunakan metode penentuan awal yang lebih canggih (seperti K-Means++) untuk hasil yang lebih stabil.

2. Iterasi (Pengulangan):

- Lakukan langkah-langkah berikut secara berulang sampai kondisi berhenti terpenuhi:
 - a. Langkah Penugasan (Assignment Step): Untuk setiap objek data (x_i): Hitung jarak (biasanya Jarak Euclidean) dari objek x_i ke setiap K sentroid (μ_j). Tugaskan objek x_i ke kluster j yang memiliki sentroid μ_j dengan jarak terdekat.
 - b. Langkah Pembaruan (Update Step): Untuk setiap kluster j (dari 1 sampai K): Hitung ulang posisi sentroid μ_j . Sentroid baru adalah nilai rata-rata (mean) dari semua objek data yang saat ini ditugaskan ke kluster j . Jika sebuah kluster tidak memiliki anggota (ini jarang terjadi tapi mungkin), sentroidnya bisa dibiarkan, dihapus, atau diinisialisasi ulang.

K-Means Algoritma

3. Kondisi Berhenti (Stopping Condition):

- Ulangi Langkah 2 sampai salah satu kondisi berikut terpenuhi:
 - Konvergensi: Tidak ada (atau sangat sedikit) objek data yang berpindah kluster antara iterasi saat ini dan iterasi sebelumnya.
 - Sentroid Stabil: Posisi sentroid tidak berubah (atau berubah sangat sedikit) antara iterasi.
 - Jumlah Iterasi Maksimum: Telah mencapai batas jumlah pengulangan yang ditentukan sebelumnya.

K-Means - Ilustrasi

Objek	X1	X2	X3
O1	1	2	1
O2	2	1	2
O3	8	9	8
O4	9	8	9
O5	18	19	18

K-Means - Ilustrasi

Langkah 1: Inisialisasi (k=2)

Ditentukan jumlah kluster $k=2$. Pilih pusat kluster (*centroid*) awal secara acak. Misal diambil:

- Centroid 1 (M_1): Objek O1 = (1, 2, 1)
- Centroid 2 (M_2): Objek O3 = (8, 9, 8)

Iterasi 1

A. Penghitungan Jarak ke Centroid (Euclidean)

Hitung jarak setiap objek O_i ke M_1 dan M_2 .

$$d(O_i, M_j) = \sqrt{(X_{i1} - M_{j1})^2 + (X_{i2} - M_{j2})^2 + (X_{i3} - M_{j3})^2}$$

- O1 ke M_1 : $\sqrt{(1 - 1)^2 + (2 - 2)^2 + (1 - 1)^2} = 0$
- O1 ke M_2 : $\sqrt{(1 - 8)^2 + (2 - 9)^2 + (1 - 8)^2} = \sqrt{49 + 49 + 49} = 12.12$
- O5 ke M_1 : $\sqrt{(18 - 1)^2 + (19 - 2)^2 + (18 - 1)^2} = \sqrt{289 + 289 + 289} = 29.44$
- O5 ke M_2 : $\sqrt{(18 - 8)^2 + (19 - 9)^2 + (18 - 8)^2} = \sqrt{100 + 100 + 100} = 17.32$
- Lakukan untuk seluruh kombinasi.

K-Means - Ilustrasi

Matriks Jarak dan Alokasi Kluster Iterasi 1

Objek	Jarak ke M1	Jarak ke M2	Jarak Terdekat	Kluster
O1	0	12.12	0	K1
O2	1.73	11.66	1.73	K1
O3	12.12	0	0	K2
O4	12.81	1.73	1.73	K2
O5	29.44	17.32	17.32	K2

K-Means - Ilustrasi

Iterasi 2

A. Pembaruan Centroid

Hitung rata-rata posisi objek dalam masing-masing kluster baru.

Kluster 1 {O1, O2}:

$$M_{1,baru} = \left(\frac{1+2}{2}, \frac{2+1}{2}, \frac{1+2}{2} \right) = (1.5, 1.5, 1.5)$$

Kluster 2 {O3, O4, O5}:

$$M_{2,baru} = \left(\frac{8+9+18}{3}, \frac{9+8+19}{3}, \frac{8+9+18}{3} \right) = (11.67, 12, 11.67)$$

K-Means - Ilustrasi

A. Penghitungan Jarak ke Centroid (Euclidean)

Hitung jarak setiap objek O_i ke M_1 dan M_2 .

$$d(O_i, M_j) = \sqrt{(X_{i1} - M_{j1})^2 + (X_{i2} - M_{j2})^2 + (X_{i3} - M_{j3})^2}$$

- O1 ke M_1 : $\sqrt{(1 - 1.5)^2 + (2 - 1.5)^2 + (1 - 1.5)^2} = \sqrt{0.25 + 0.25 + 0.25} = \sqrt{0.75} = 0.87$
- O1 ke M_2 : $\sqrt{(1 - 11.67)^2 + (2 - 12)^2 + (1 - 11.67)^2} = \sqrt{113.85 + 100 + 113.85} = \sqrt{327.70} = 18.10$
- O5 ke M_1 : $\sqrt{(18 - 1.5)^2 + (19 - 1.5)^2 + (18 - 1.5)^2} = \sqrt{272.25 + 306.25 + 272.25} = \sqrt{850.75} = 29.17$
- O5 ke M_2 : $\sqrt{(18 - 11.67)^2 + (19 - 12)^2 + (18 - 11.67)^2} = \sqrt{40.07 + 49 + 40.07} = \sqrt{129.14} = 11.36$
- Lakukan untuk kombinasi lainnya

K-Means - Ilustrasi

Objek	Jarak ke M1	Jarak ke M2	Jarak Terdekat	Klaster
O1	0.87	18.10	0.87	K1
O2	0.87	17.80	0.87	K1
O3	11.26	6.00	6.00	K2
O4	12.99	5.39	5.39	K2
O5	29.17	11.36	11.36	K2

Karena alokasi objek pada **Iterasi 2** sama persis dengan **Iterasi 1** (O1, O2 di K1; O3, O4, O5 di K2), maka proses dinyatakan **Konvergen**.

K-Means - Ilustrasi

Hitung SSE nya :

Penghitungan SSE Klaster 1 (K1)

Anggota: {O1, O2} | Centroid M_1 : (1.5, 1.5, 1.5)

- Kuadrat Jarak O1: $(1 - 1.5)^2 + (2 - 1.5)^2 + (1 - 1.5)^2 = (-0.5)^2 + (0.5)^2 + (-0.5)^2 = 0.25 + 0.25 + 0.25 = 0.75$
- Kuadrat Jarak O2: $(2 - 1.5)^2 + (1 - 1.5)^2 + (2 - 1.5)^2 = (0.5)^2 + (-0.5)^2 + (0.5)^2 = 0.25 + 0.25 + 0.25 = 0.75$
- WCSS K1: $0.75 + 0.75 = 1.5$

Penghitungan SSE Klaster 2 (K2)

Anggota: {O3, O4, O5} | Centroid M_2 : (11.67, 12, 11.67)

- Kuadrat Jarak O3: $(8 - 11.67)^2 + (9 - 12)^2 + (8 - 11.67)^2 = (-3.67)^2 + (-3)^2 + (-3.67)^2 \approx 35.94$
- Kuadrat Jarak O4: $(9 - 11.67)^2 + (8 - 12)^2 + (9 - 11.67)^2 = (-2.67)^2 + (-4)^2 + (-2.67)^2 \approx 30.26$
- Kuadrat Jarak O5: $(18 - 11.67)^2 + (19 - 12)^2 + (18 - 11.67)^2 = (6.33)^2 + (7)^2 + (6.33)^2 \approx 129.14$
- WCSS K2: $35.94 + 30.26 + 129.14 = 195.34$

SSE TOTAL

$$Total\ SSE = WCSS\ K1 + WCSS\ K2$$

$$Total\ SSE = 1.5 + 195.34 = 196.84$$

K-Means - R

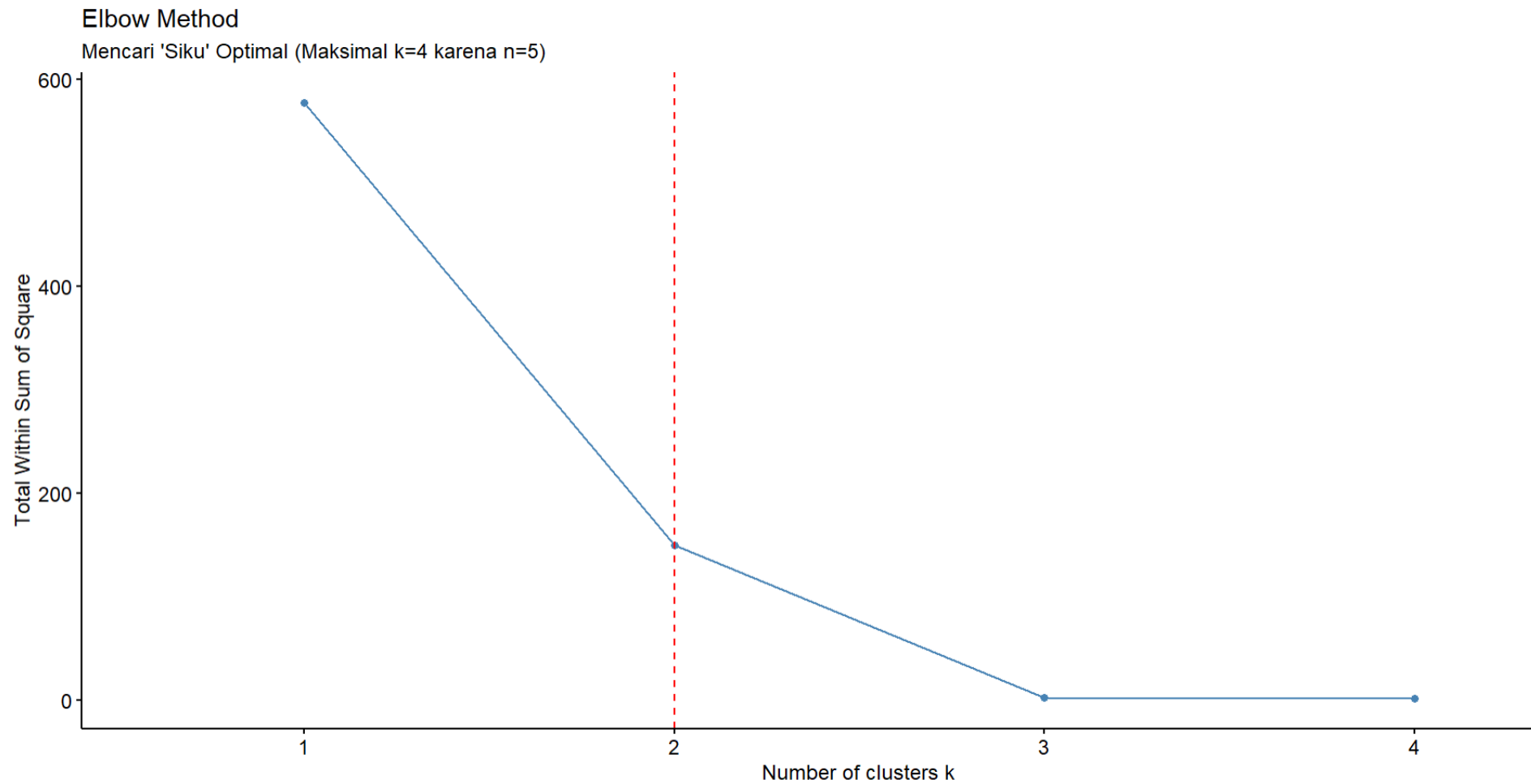
```
library(factoextra)
# Data
data_kmeans <- data.frame( x1 = c(1, 2, 8, 9, 18), x2 = c(2, 1, 9, 8, 19), x3 = c(1, 2, 8, 9, 18))
rownames(data_kmeans) <- paste0("O", 1:5)
# Elbow Method (DIBERIKAN k.max = 4)
fviz_nbclust(data_kmeans, kmeans, method = "wss", k.max = 4) + geom_vline(xintercept = 2,
linetype = 2, col = "red") + labs(title = "Elbow Method", subtitle = "Mencari 'Siku' Optimal
(Maksimal k=4 karena n=5)")
# Eksekusi K-Means k=2
set.seed(123)
hasil_km <- kmeans(data_kmeans, centers = 2, nstart = 25)
# Visualisasi Klustering
fviz_cluster(hasil_km, data = data_kmeans, palette = "jco", ellipse.type = "convex",
ggtheme = theme_minimal())
hasil_km$centers # Menampilkan Centroid Akhir
hasil_km$tot.withinss # Menampilkan Total SSE (Withinss)
hasil_km$cluster # Menampilkan Vektor Anggota Klaster
# Menampilkan List Anggota per Klaster
split(rownames(data_kmeans), hasil_km$cluster)
```

K-Means – R

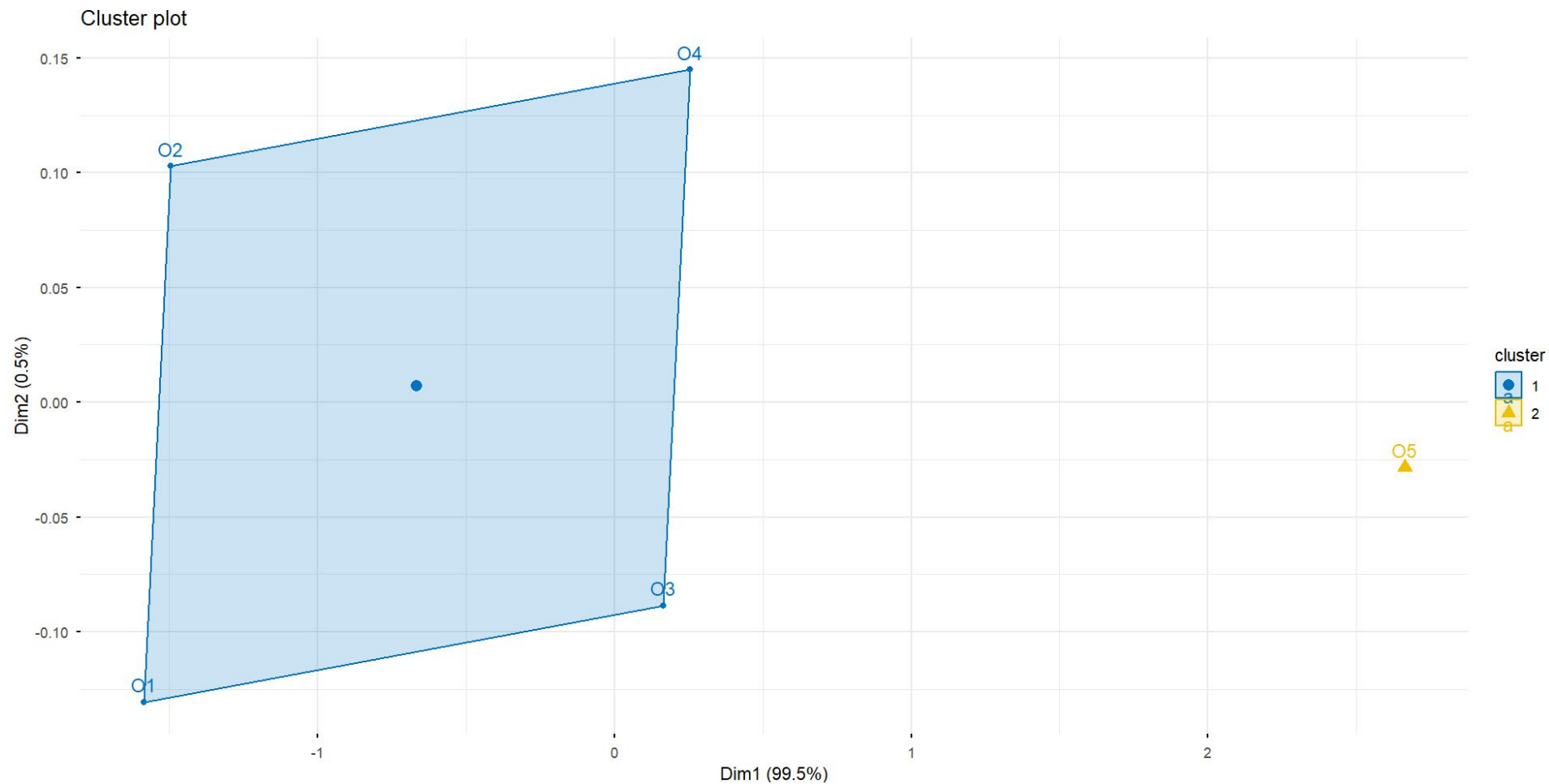
```
> hasil_km$centers      # Menampilkan Centroid Akhir
  x1 x2 x3
1  5  5  5
2 18 19 18
> hasil_km$tot.withinss # Menampilkan Total SSE (Withinss)
[1] 150
> hasil_km$cluster      # Menampilkan Vektor Anggota Klaster
01 02 03 04 05
 1  1  1  1  2
> # Menampilkan List Anggota per Klaster
> split(rownames(data_kmeans), hasil_km$cluster)
$`1`
[1] "01" "02" "03" "04"

$`2`
[1] "05"
```

K-Means - R



K-Means - R



K-Medoids Algoritma

K-Medoids, atau juga dikenal sebagai algoritma PAM (Partitioning Around Medoids), adalah metode pengelompokan non-hierarki yang merupakan pengembangan dari algoritma K-Means. Perbedaan mendasar terletak pada cara penentuan pusat kluster, di mana K-Medoids menggunakan objek nyata sebagai pusat, bukan nilai rata-rata.

Konsep Dasar: Medoid vs Centroid

- Centroid (K-Means): Pusat kluster adalah nilai rata-rata (*mean*) dari seluruh anggota kluster. Titik ini biasanya merupakan titik imajiner yang tidak ada dalam dataset asli.
- Medoid (K-Medoids): Pusat kluster adalah salah satu objek yang benar-benar ada di dalam dataset tersebut. Objek ini dipilih karena memiliki ketidaksamaan (*dissimilarity*) paling minimum terhadap semua objek lain dalam klasternya.

K-Medoids Algoritma

K-Medoids sering dipilih sebagai alternatif K-Means karena beberapa alasan teknis berikut:

1. Robust terhadap Outlier: K-Means sangat sensitif terhadap pencilan (*outlier*) karena nilai rata-rata akan tertarik ke arah angka ekstrem. K-Medoids lebih stabil karena menggunakan objek representatif (medoid).
2. Fleksibilitas Jarak: K-Medoids dapat digunakan dengan berbagai ukuran jarak (seperti *Manhattan Distance*) yang lebih cocok untuk data dengan variansi tinggi, sedangkan K-Means sangat bergantung pada *Euclidean Distance*.
3. Akurasi pada Data Kecil: Cenderung lebih efektif untuk dataset berukuran kecil hingga menengah.

K-Medoids Algoritma

•Silhouette Method: Memaksimalkan Rasio Kebaikan Klaster. Dia membandingkan seberapa rapat objek di kelompoknya sendiri (a) dibandingkan seberapa jauh dia dari kelompok tetangganya (b).

Rumus Silhouette Coefficient (s):

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Di mana:

- $a(i)$: Rata-rata jarak objek ke teman-teman se-klasternya. (Makin kecil makin bagus).
- $b(i)$: Rata-rata jarak objek ke objek di klaster tetangga terdekat. (Makin besar makin bagus).

K-Medoids Algoritma

1. Inisialisasi (BUILD Step):

- Pilih K: Tentukan jumlah kluster yang diinginkan.
- Pilih Medoid Awal: Pilih K objek data dari dataset secara acak (atau dengan metode heuristik) untuk menjadi medoid awal.

2. Langkah Penugasan (Assignment Step):

- Untuk setiap objek data yang bukan medoid:
 - Hitung jaraknya ke setiap K medoid.
 - Tugaskan objek tersebut ke kluster yang medoidnya paling dekat.

K-Medoids Algoritma

3. Langkah Pembaruan (SWAP Step - Iteratif):

- Ini adalah inti dari PAM. Tujuannya adalah mencoba mengganti medoid saat ini dengan non-medoid untuk melihat apakah kualitas kluster (biasanya diukur dengan total jarak semua objek ke medoid terdekatnya) bisa ditingkatkan.
- Lakukan pengulangan sampai tidak ada perubahan (atau perubahan sangat kecil):
 - Pilih pasangan (Medoid, Non-Medoid): Pilih satu medoid saat ini (m_i) dan satu objek non-medoid (o_j) dari dataset.
 - Hitung Biaya Swap: Hitung "biaya" total (total jarak) jika o_j dijadikan medoid baru menggantikan m_i . Ini melibatkan perhitungan bagaimana objek-objek lain akan ditugaskan ulang jika o_j menjadi medoid.
 - Evaluasi Swap: Bandingkan biaya total setelah swap dengan biaya total sebelum swap.
 - Lakukan Swap (Jika Menguntungkan): Jika biaya total menurun setelah swap, maka jadikan o_j sebagai medoid baru dan m_i menjadi objek non-medoid biasa. Tugaskan kembali objek-objek yang terpengaruh.
 - Ulangi: Coba semua kemungkinan pasangan (Medoid, Non-Medoid) atau gunakan strategi tertentu untuk memilih pasangan yang akan dicoba. Lanjutkan proses ini sampai tidak ada lagi swap yang bisa menurunkan biaya total.
 - Setelah tidak ada lagi swap yang menguntungkan, K objek yang menjadi medoid saat itu adalah medoid akhir, dan penugasan objek ke kluster tersebut adalah hasil klasteringnya.

K-Medoids Algoritma - Ilustrasi

Objek	X1	X2	X3
O1	1	2	1
O2	2	1	2
O3	8	9	8
O4	9	8	9
O5	18	19	18

Jarak Manhattan

$$d(i, j) = |X_{i1} - X_{j1}| + |X_{i2} - X_{j2}| + |X_{i3} - X_{j3}|$$

Inisialisasi (k=2)

Kita pilih medoid awal, misal:

- Medoid 1 (M_1): O1 (1, 2, 1)
- Medoid 2 (M_2): O3 (8, 9, 8)

K-Medoids Algoritma - Ilustrasi

Penghitungan Biaya (Total Cost)

A. Contoh Hitung Jarak Manhattan:

- O1 ke O2: $|1 - 2| + |2 - 1| + |1 - 2| = 1 + 1 + 1 = 3$
- O1 ke O3: $|1 - 8| + |2 - 9| + |1 - 8| = 7 + 7 + 7 = 21$
- O3 ke O4: $|8 - 9| + |9 - 8| + |8 - 9| = 1 + 1 + 1 = 3$
- O3 ke O5: $|8 - 18| + |9 - 19| + |8 - 18| = 10 + 10 + 10 = 30$
- Hitung untuk kombinasi lainnya

Total Cost (S_1):

$$S_1 = 0 + 3 + 0 + 3 + 30 = 36$$

Objek	Jarak ke M1(O1)	Jarak ke M2(O3)	Jarak Terdekat	Klaster
O1	0	21	0	K1
O2	3	20	3	K1
O3	21	0	0	K2
O4	22	3	3	K2
O5	51	30	30	K2

K-Medoids Algoritma - Ilustrasi

Iterasi 2: Proses Pertukaran (Swap O3 dengan O5)

Kita uji apakah menukar pusat dari O3 ke O5 akan membuat total jarak lebih pendek.

- Medoid Baru 1 (M_1): O1 (1, 2, 1)
- Medoid Baru 2 (M_2): O5 (18, 19, 18)

A. Penghitungan Jarak ke Medoid Baru:

- O5 ke O4: $|18 - 9| + |19 - 8| + |18 - 9| = 9 + 11 + 9 = 29$
- O1 ke O4: 22 (dari hitungan sebelumnya)
- Lakukan ke seluruh kombinasi SWAP

Total Cost Baru (S_2):

$$S_2 = 0 + 3 + 21 + 22 + 0 = 46$$

Karena $46 > 36$, maka pertukaran ditolak. Konfigurasi terbaik tetap menggunakan O1 dan O3 sebagai medoid.

Objek	Jarak ke M1(O1)	Jarak ke M2(O5)	Jarak Terdekat	Klaster
O1	0	51	0	K1
O2	3	48	3	K1
O3	21	30	21	K1
O4	22	29	22	K1
O5	51	0	0	K2

K-Medoids Algoritma - Ilustrasi

Untuk menentukan jumlah kemungkinan pertukaran (*swap*) dalam satu putaran penuh, rumusnya sederhana:

$$\text{Jumlah Swap} = k \times (n - k)$$

Di mana:

- k = Jumlah Medoid (Klaster)
- $n - k$ = Jumlah Objek yang **bukan** Medoid

Jika kita punya 5 objek (O_1, O_2, O_3, O_4, O_5) dan 2 di antaranya sudah jadi Medoid (misal O_1 dan O_3), maka:

- Medoid (k): O_1 dan O_3 (Ada 2)
- Non-Medoid ($n-k$): O_2, O_4, O_5 (Ada 3)

Maka, total kombinasi pertukaran yang harus dicoba algoritma adalah:

$$2 \times 3 = 6 \text{ **kemungkinan**}$$

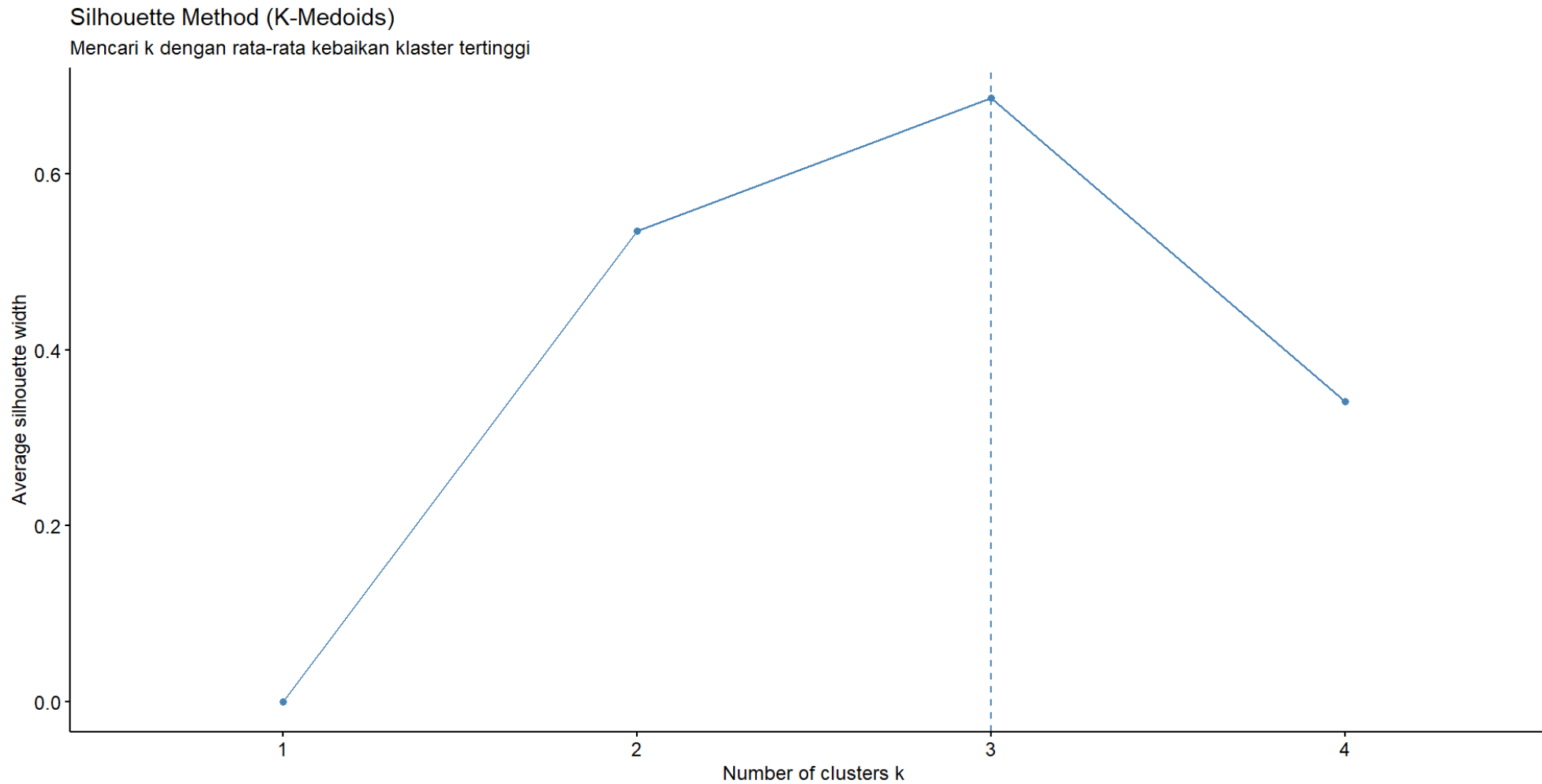
Rincian 6 Kemungkinan Tersebut:

1. Tukar O_1 dengan O_2
2. Tukar O_1 dengan O_4
3. Tukar O_1 dengan O_5
4. Tukar O_3 dengan O_2
5. Tukar O_3 dengan O_4
6. Tukar O_3 dengan O_5 (Ini yang tadi disimulasikan dan hasilnya ditolak)

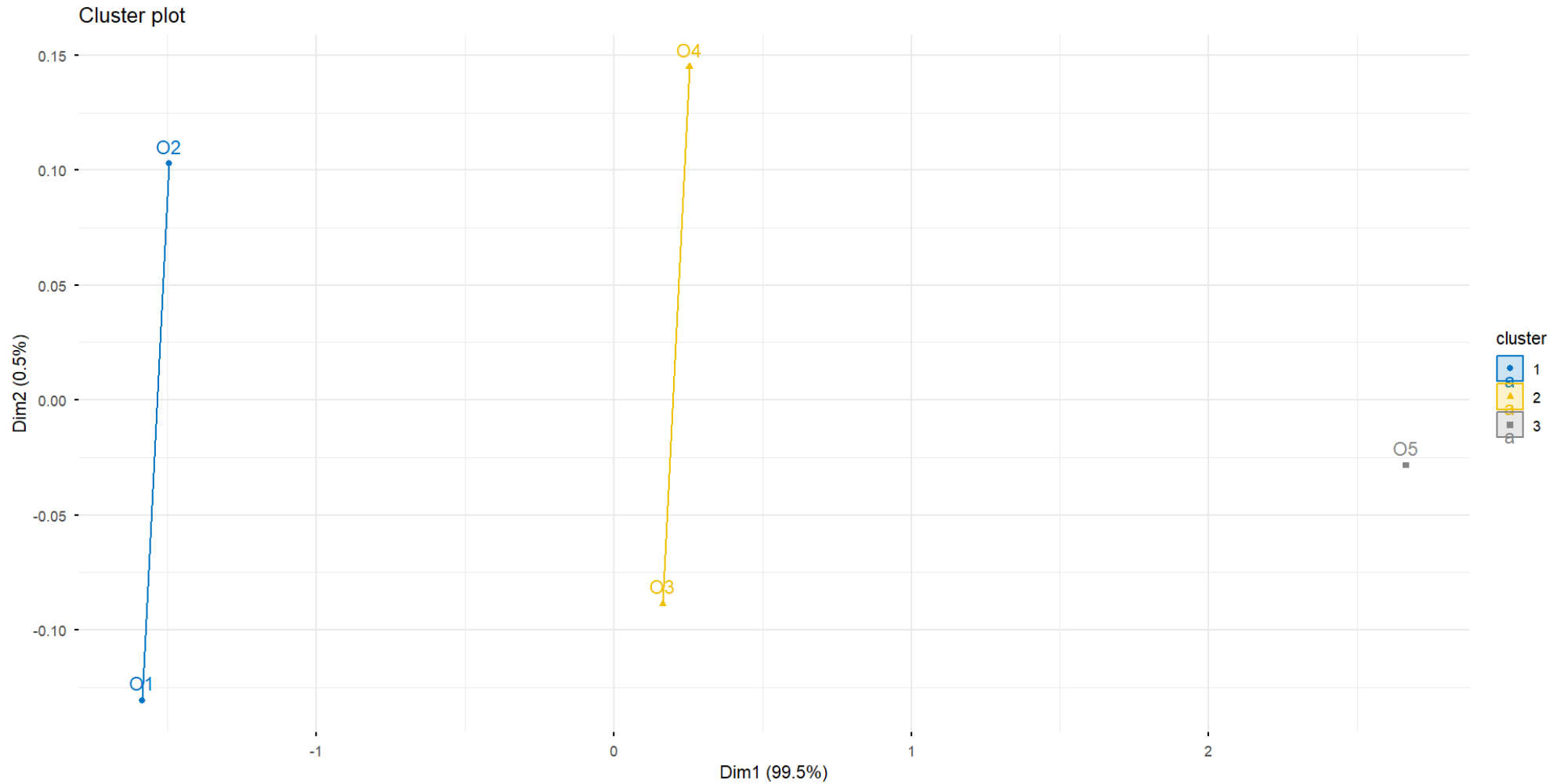
K-Medoids Algoritma - R

```
#KMEDOID
library(cluster)
library(factoextra)
# Persiapan Data
data_kmedoids <- data.frame( x1 = c(1, 2, 8, 9, 18), x2 = c(2, 1, 9, 8, 19), x3 = c(1, 2, 8, 9, 18))
rownames(data_kmedoids) <- paste0("O", 1:5)
# Visualisasi Silhouette Method
# Menggunakan fungsi pam untuk mencari k optimal
fviz_nbclust(data_kmedoids, pam, method = "silhouette", k.max = 4) + labs(title = "Silhouette Method (K-Medoids)",
  subtitle = "Mencari k dengan rata-rata kebaikan kluster tertinggi")
# Eksekusi K-Medoids (PAM) k=3
# metric = "manhattan" (atau "euclidean")
hasil_pam <- pam(data_kmedoids, k = 3, metric = "manhattan")
#Visualisasi Klustering
fviz_cluster(hasil_pam, data = data_kmedoids, palette = "jco", ellipse.type = "convex", ggtheme =
  theme_minimal())#
#Menampilkan Output Penting
hasil_pam$medoids # Menampilkan Objek yang Jadi Medoid Akhir
hasil_pam$objective[2] # Menampilkan Total Cost (Build + Swap)
hasil_pam$clustering # Menampilkan Vektor Anggota Kluster
# Menampilkan List Anggota per Klustersplit(rownames(data_kmedoids), hasil_pam$clustering)
```

K-Medoids Algoritma - R



K-Medoids Algorithm - R



K-Medoids Algoritma - k=2 atau 3?

Jika pilih k=2

(Klaster 1: $\{O_1, O_2\}$ / Klaster 2: $\{O_3, O_4, O_5\}$)

Mari kita hitung sampel objek yang dipaksakan di k=2, yaitu O_3 :

- Hitung $a(O_3)$: Jarak ke teman seklaster (O_4 dan O_5).

- $a(O_3) = \frac{d(O_3, O_4) + d(O_3, O_5)}{2} = \frac{3 + 30}{2} = \mathbf{16.5}$

- Hitung $b(O_3)$: Jarak ke klaster tetangga (Klaster 1: $\{O_1, O_2\}$).

- $b(O_3) = \frac{d(O_3, O_1) + d(O_3, O_2)}{2} = \frac{21 + 20}{2} = \mathbf{20.5}$

- Hitung $s(O_3)$:

- $s(O_3) = \frac{20.5 - 16.5}{20.5} = \mathbf{0.19}$

Kesimpulan k=2: Nilai 0.19 itu sangat rendah (mendekati 0), artinya O_3 sebenarnya merasa "salah tempat" karena dipaksa satu kelompok dengan O_5 yang jaraknya sangat jauh (30). Ini yang membuat rata-rata total di grafik hanya 0.53.

Jika pilih k=3

(Klaster 1: $\{O_1, O_2\}$ / Klaster 2: $\{O_3, O_4\}$ / Klaster 3: $\{O_5\}$)

Sekarang kita lihat bagaimana O_3 di k=3:

- Hitung $a(O_3)$: Sekarang temannya Cuma O_4 .

- $a(O_3) = d(O_3, O_4) = \mathbf{3}$

- Hitung $b(O_3)$: Jarak ke tetangga terdekat (Klaster 1: $\{O_1, O_2\}$).

- $b(O_3) = \frac{d(O_3, O_1) + d(O_3, O_2)}{2} = \frac{21 + 20}{2} = \mathbf{20.5}$

- Hitung $s(O_3)$:

- $s(O_3) = \frac{20.5 - 3}{20.5} = \frac{17.5}{20.5} = \mathbf{0.85}$

Kesimpulan k=3: O_3 Naik drastis, Karena sudah tidak sekelompok dengan O_5 . Ini yang membuat rata-rata total di grafik naik menjadi 0.69.

DBSCAN Algoritma

1. Pemilihan Parameter: Tentukan nilai eps dan MinPts. Pemilihan ini krusial dan seringkali memerlukan eksperimen atau pengetahuan domain.
2. Inisialisasi: Tandai semua titik sebagai "belum dikunjungi" (unvisited).
3. Iterasi: Mulai dari titik acak yang belum dikunjungi:
 - Tandai: Tandai titik tersebut sebagai "telah dikunjungi".
 - Cek Lingkungan: Temukan semua titik tetangga yang berada dalam jarak eps dari titik saat ini.
 - Evaluasi Kepadatan:
 - Jika jumlah tetangga (termasuk titik itu sendiri) $\geq$ MinPts:
 - Titik ini adalah Core Point.
 - Buat kluster baru (jika titik ini belum termasuk kluster mana pun).
 - Tambahkan titik ini ke kluster baru.
 - Tambahkan semua tetangganya ke dalam "daftar antrian" untuk diproses.

DBSCAN Algoritma

- Jika jumlah tetangga $< \text{MinPts}$:
 - Titik ini (untuk sementara) ditandai sebagai Noise Point. (Ia mungkin akan menjadi Border Point nanti jika ditemukan bertetangga dengan Core Point lain).

4. Ekspansi Klaster: Selama "daftar antrian" tidak kosong:

- Ambil satu titik (P') dari antrian.
- Jika P' belum dikunjungi: Tandai P' sebagai "telah dikunjungi". Cek lingkungan P' . Jika P' juga Core Point (memiliki $\geq \text{MinPts}$ tetangga), tambahkan semua tetangganya ke dalam "daftar antrian".
- Jika P' belum menjadi anggota klaster mana pun, tambahkan P' ke klaster saat ini.

5. Ulangi: Kembali ke Langkah 3, pilih titik lain yang belum dikunjungi, dan ulangi prosesnya sampai semua titik telah dikunjungi dan diklasifikasikan (baik sebagai bagian dari klaster atau sebagai noise).

CLARA Algoritma

1. Inisialisasi: Set `min_cost` (biaya minimum) ke nilai yang sangat besar.
2. Loop Pengambilan Sampel: Ulangi sebanyak `S` kali:
 - a. Ambil Sampel: Pilih secara acak `s` objek dari dataset `X` untuk membentuk `current_sample`.
 - b. Jalankan PAM: Terapkan algoritma PAM pada `current_sample` untuk mendapatkan `K` medoid (`current_medoids`).
 - c. Hitung Biaya: Hitung biaya total (biasanya rata-rata jarak/ketidakmiripan) dengan cara menugaskan setiap objek dalam seluruh dataset `X` ke medoid terdekat dari `current_medoids`. Simpan biaya ini sebagai `current_cost`.
 - d. Bandingkan & Simpan: Jika `current_cost < min_cost`: Perbarui `min_cost = current_cost`. Simpan `current_medoids` sebagai `best_medoids`.
3. Output: Setelah loop selesai, `best_medoids` adalah set medoid akhir yang dipilih oleh CLARA.
4. Penugasan Akhir (Opsional tapi Umum): Tugaskan setiap objek dalam dataset `X` ke medoid terdekat dari `best_medoids` untuk mendapatkan partisi kluster akhir.